
Design Project 3 – Sense of Independence

Mr. Muscles – Handling Muscle Atrophy One Step at a Time

IBEHS 1P10 – Health Solutions Design Projects

Tutorial T03

Team 22

MATTHEW HO (hom23)

MAYAR ALJAYOUSH (aljayoum)

RUIDI LIU (liu127)

SHARANYA SRIRANGAM (srirans)

Submitted: March 7th, 2021

Table of Contents

Academic Integrity Statement	3
Executive Summary	4
Market Analysis	5
Product #1	5
Product #2	5
Description of Device Design (Design Specifications).....	6
Project Schedule.....	7
Preliminary Gantt Chart	7
Final Gantt Chart	7
Logbook of Additional Meetings and Discussions	8
Scheduled Weekly Agendas and Meeting Minutes	16
January 20	16
January 27	16
February 3	16
February 17	16
February 24	16
March 3	16
Design Studio Worksheets.....	16
Milestone 0.....	16
Milestone 1.....	16
Milestone 2.....	16
Milestone 3.....	16
Milestone 4.....	16
List of Sources	17
Source Materials Database	17
References	18
Appendices.....	19
Appendix A: Autodesk CAD Modelling.....	19
Appendix B: Computer Program	41
Appendix C: Demonstration of the Computer program.....	52

Academic Integrity Statement

The student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University.

Mayar Aljayoush 400324172

X 

The student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University.

Matthew Ho 400303697

X 

The student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University.

Ruidi Liu 400296859

X 

The student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University.

Sharanya Srirangam 400315931

X 

Executive Summary

Muscle atrophy is the degradation of skeletal muscles over time until they no longer function properly. Sarcopenia is a type of muscle atrophy defined as the age-related, involuntary loss of skeletal muscle mass and strength. By the 8th decade of life, up to 50% of muscle mass is lost due to sarcopenia, and the pathological effects due to this adverse change on the body may result in disability, frailty, and loss of function and independence [1]. On average, it is estimated that 5–13% of elderly people aged 60–70 years are affected by sarcopenia and rises to 11–50% for ages 80 and above [2]. Pathological effects that cause sarcopenia include age-related molecular change, hormonal change, and environmental causes such as decreased activity and nutrition [1]. Although molecular and hormonal change can be difficult to combat, environmental changes can be targeted to decrease the effects of atrophy, namely exercise and muscle activity. Aerobic, resistance and combined exercises are the most beneficial in decreasing the pathological effects of sarcopenia, and our device intends to promote consistent muscular exercise to decrease atrophy effects in elderly patients [3].

The proposed design solution is the Mr. Muscles EMG band, a wearable personal monitoring device for elderly patients that are suffering from sarcopenia (figure 1.A2). This EMG band is targeted towards patients who are receiving rehabilitation treatment by a physician, through an exercise plan allowing for the recovery of their muscle mobility. It allows for remote monitoring of a patient's EMG voltages in a certain muscle group by emailing information to the physician. It also reminds the patient to exercise with 2 different output devices: a buzzer on the EMG band and a text message sent to the patient. The objectives of this device were to take in continuous input of EMG voltage values and calculate the patient's period of rest between exercise sessions so that they can receive reminders to exercise as elders tend to be forgetful [4]. Moreover, it enables the physician to monitor the patient's status and modify the exercise plan according to the data by relaying information.

The user could set up the EMG band with a given resting time interval, normal EMG, and active EMG voltage value from the physician. Normal EMG value is the voltage of the electric signals inside a muscle at a normal state whereas the active EMG value is the standard voltage of the electric signals when the users are exercising. Once the patient's voltages are lower than the active EMG value, the program inside the EMG band starts timing. If the patient has rested longer than the prescribed resting time, the buzzer turns on and a text message will be sent to the user's phone, reminding them to exercise again. The buzzer will turn off once the user's voltages are greater than the active EMG value. Besides, if the user has abnormal voltages lower than the normal EMG values, an email will be sent to the physician, alerting them to check their patient's data.

Market Analysis

Product #1

Mbody 3 shorts is one of the products of the Mbody series produced by Myontec, cutting-edge wearables company that uses EMG, to understand muscles behavior. The Mbody 3 products are designed as comprehensive wireless tools to observe muscles during exercising. Mshorts 3 measures data from hamstrings, quadriceps, and glutes with the help of Mcell 3, a small, lightweight, and easy-to-use device attached to the product with EMG and inertial sensory [5] [6]. The recorded data could be accessed through either a mobile application or a muscle monitor software [7].

Similarities include that they are both personal wearable devices that could detect and monitor muscle activation of the thighs by using EMG sensors. Both products could deliver monitoring data and provide feedback to the user.

A major difference between the two products is that the Mshorts 3 also includes detecting and analyzing muscle imbalance and relaxation function [7]. It can also show the muscle loading distribution on the mobile application or computer software while the EMG band could only output the data as a text file to be sent to the user [7]. On the other hand, Mshorts 3 is fully machine washable. It can be cleaned in the washing machine at 30 °C while the EMG band cannot remove the Spandex band to wash [7]. However, the EMG band can remind users to exercise and send emails to physicians when needed whereas Mshorts 3 does not have this function.

Product #2

Shimmer3 EMG Unit is an EMG sensor that measures electrical muscle activity through a non-invasive wearable device. It is a wireless solution for muscle, gait, and posture data analysis for users, containing an ECG functionality [8]. The product consists of the sensor itself but can be strapped onto any part of the body.

One of the similarities between the Dr. Muscles EMG band and the Shimmer3 EMG Unit, is that both products contain sensors that are used to measure the voltage in muscles [8]. They can be worn as a band and used in rehabilitation therapy or for exercise. The two products are non-invasive and represent the activity of the whole muscle group [8]. Moreover, they provide wireless communication of the data as an output.

Moving onto the differences, the Shimmer3 EMG Unit is a multi-use sensor, purely for outputting the voltage data, while Mr. Muscles EMG band has a far more specific focus. The EMG band is specifically designed to target the thigh muscles of elderly patients suffering from muscle atrophy. The programming associated with the EMG band contains even more features such as calculating the period of rest, reminding the user to exercise again, sending text messages, and emailing the physician about abnormal muscle activity.

Moreover, the Shimmer3 product can be used for any part of the body but does not come with a band to secure it [8].

Description of Device Design (Design Specifications)

The Mr. Muscles EMG Band is a device that is wrapped around the upper thigh to monitor the muscle activity levels of the quadricep or hamstring muscles. The device consists of a small housing component that contains various devices that cumulatively measure muscle activity levels, and an elastic band that wraps around the thigh to secure the housing in place. Refer to figure 2.A2.

The housing component contains multiple devices, dissected in figure 4.A2. The MyoWare EMG muscle sensor uses 2 electrodes placed on the muscle body and in the muscle length direction, while protruding through the casing to come into direct contact with the skin. The Arduino Nano RP2040 Connect device serves as the WiFi connection between the EMG sensor and the Raspberry Pi [9]. Since the EMG sensor consistently requires external Python programming to operate, the RP2040 wirelessly connects the EMG sensor to the Raspberry Pi. The AD/DA converter serves as a translator between the MyoWare EMG sensor and the RP2040, to properly relay information. The buzzer is in direct contact with the skin, to ensure the user can feel the vibrational queue when alerts are sent. The RAVPower 2800mAh Li-ion Battery is the power source for all components in the EMG band. [10] This battery is suitable because this model is used in various android smartphones. Thus, it has the necessary battery life, voltage output, physical size, and heat management to operate in this device.

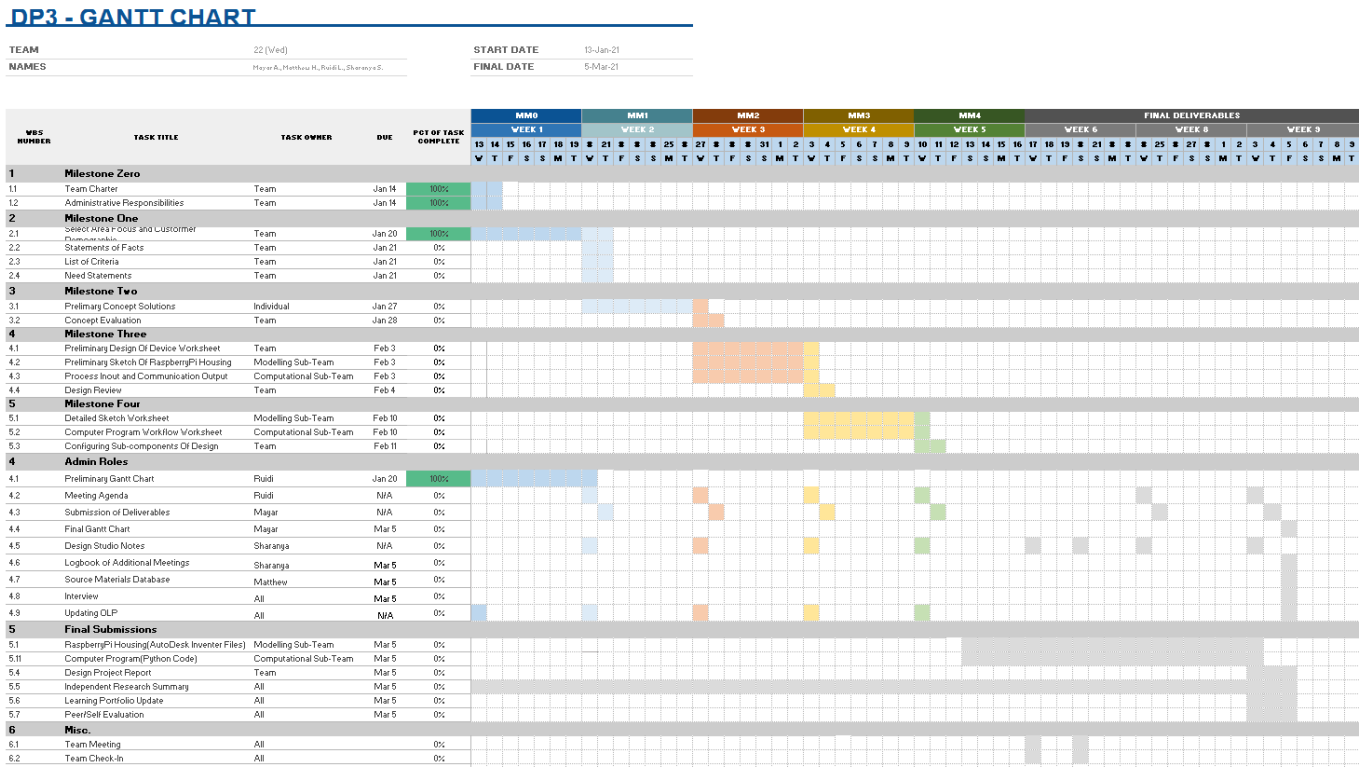
Our device consists of 2 layers, because applying all the components in one-layer results in an excessively long design (≈ 20 cm), which would interfere with elderly body mobility. Using 2 layers, the length is decreased without sacrificing function or body mobility. In the second layer, a small gap at the rear has been included to allow for wires to connect between the layers. This creates better wire management and appropriate connection capabilities. The second layer is held in place by rails on each side to prevent vertical translation. The first layer necessarily contains the EMG sensor and buzzer as they encounter the skin, while the other components are organized for optimal arrangement.

The lid is located at one end of the housing and is secured in place by 2 Velcro straps. An access hole is located on the lid to align with the battery charging port, so charging does not require opening the lid. The lid is meant to stay closed and is only operated as a last resort for interior maintenance or replacements.

The band is made of a Spandex stretch fabric, with high durability and wear [11]. A Velcro patch is attached at the tip of the band to secure the band in place as shown in figure 3.A2. The band is permanently attached to a cylindrical lug on one end and is impermanently wrapped around a lug on the other end. Pulling the strap on the open lug tightens the rest of the band, similar to the mechanism used in watches.

Project Schedule

Preliminary Gantt Chart



Final Gantt Chart

Rehab Gym Users (at-home workouts)	Wearable Design	Measuring heart rate during physical activity	Pulse/Force/EMG	maybe avoid because it is pretty common, and we used it for the mini activity
patients that had a spine surgery/ implant	Wearable Design	measure the muscles contractions and rate of healing	Emg/ Force	
Any patient that had a surgery of an organ removal/implant/ open heart/broken bones surgeries	Wearable Design	check on the appearance of post-surgical inflammations and control that inflammation before it extremes	Thermal/pulse	
Patients with weak grips (had wrist joint replacement surgeries or wrist surgeries) (Elderly Maybe?)	Wearable Design	Measures the grip strength of the patient when practicing	Force Sensitive Sensor	☐ maybe design some sort of glove/cuff ☐ wouldn't this limit the design to a specific palm-size ? ☐ EMG - measures the force of contraction. ☐ Force Sensitive - measure the force of muscle.
Wheelchair Users + vision loss	Wearable Design Or Home automation system device?	Point out where things are at home	Orientation sensor Distance sensor	
Vision Loss Patients	Wearable device	Where the rooms are	Distance sensor Sense walls	
Wheelchair users	Home Automation?	Allows them to reach things that are too high up	Distance sensor? Orientation sensor?	
Patients addicted to smoking (or drug overdose)	Wearable Design Or	Detects smoke or measures heart rate.	Smoke sensor or Pulse sensor	☐ Hard to make sure they will smoke at the place where the device is (Social aspect)

	Home automation	Drug Overdose/ Smoking too much symptoms include Increasing heart rate/slow erratic heart rate		☐ Take more effort to get this one work
Patients with Parkinson's Disease	Wearable Design	Tracks their tremors or shakiness	EMG sensor	☐ Glove design ☐ Monitor the data, tracking data; help physicians to make plans for the future
Patients with Rheumatoid Arthritis	Wearable Design	Assesses their inflammation using changes in temperature	Thermal sensor	Source: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6720753/ ☐ Little evidence to back up; hard to approach
People with insomnia	Wearable Design	Assesses sleep patterns and/or movement	Orientation?	hard to differentiate insomnia vs normal sleeping, difficult to get accuracy with only one sensor
Elderly patients (either recovering from surgery or are at risk to dislocations and injuries due to falling)	Wearable Design	Use orientation sensor that will alert 911 if orientation is flipped or changed abruptly (meaning the use has fallen)	Orientation sensor	challenge: already a solution BUT we can use this opportunity to make our design more unique or more compact; we do not have an accelerometer. ☐ Could be hard to design.
Muscle atrophy patients (moderate patients, not severe as they would be hospitalized)	Wearable Design	Measure muscle activity. Coding programs would calculate average activity on a scale from 1-	EMG sensor (does not included in the need statement)	☐ Could combine w/ PD idea. ☐ specifically placed on areas prone to muscle atrophy, more of a monitoring device ☐ most common in legs/arms (maybe make a band of some sort?)

		10 every certain amount of time (ex. 5 hours) to ensure that the user is activating their muscles properly		
Diabetic patients (particularly those who suffer from damaged blood vessels, in turn inhibiting sweat glands)	Wearable Design	High spikes in body temperature can be dangerous as the diabetic body cannot handle heat properly. By measuring body heat, the sensor can detect when the user needs to hydrate/cool down immediately.	Thermal sensor	☐ potential challenge: person might already know that they are feeling hot. ☐ BUT diabetic patients may not be able to feel heat rising, so could be beneficial
Anybody (?)		Detect gas leaks in the home that users may not realize	Gas sensor	

- ☐ Find the amount of muscle activity is appropriate to treat muscle atrophy!
- ☐ Ex. After 2 hours of no movement, the sensor alerts user to move! (do not interrupt sleep though)

List Of Sensors:

- ☐ EMG Sensor
- ☐ Pulse Sensor
- ☐ Force Sensitive Sensor
- ☐ Distance Sensor
- ☐ Orientation Sensor
- ☐ Gas Sensor
- ☐ Thermal Sensor

Meeting Minutes from February 16, 2021:

Time: 2:00 – 2:40 pm

What we found out from Parm: we do not need a breadboard or t-cobbler

Outputs: text message to phone, and buzzer

- we showed each other our codes
 - need to finetune the code.
 - split up the functions.
- shared the housing.
 - for a lid, try making an assembly?
 - need to make a hole for power.
- shared the band.
 - makeshift Bluetooth black box
 - include a little compartment for it.
 - Look more into the buzzer vibration mechanism.
 - make an assembly instead of just doing one part.
- shared the rubrics.
 - paid attention to details in the rubric for each subteam
 - decided that.

Coding Team: worked together to fix the rolling average.

Task Action Items:

- upload the current code that we have to teams (Ruidi)
- fix calc_time function (Sharanya)
- output device (Ruidi)
- Research the voltage values for boundaries (Ruidi)
- Write the code for a text file (final output) (Sharanya)
- Coding Team Meeting: Sunday
- Modelling Team Meeting: Friday

Meeting Minutes from February 19, 2021

Time: 7:30 -8:00 p.m.

Goals:

Discuss the design of the Raspberry Pi housing, and the individual components that need to be creatively designed with it. Discuss preliminary ideas for the EMG Band Autodesk CAD file.

Task Action Items :

Complete the Raspberry Pi housing for the meeting. Begin creating the EMG Band design as a CAD file.

Meeting Minutes from March 1, 2021

Time: 6:30 -7:00 p.m.

Goals:

Discuss the external charging ports, determine how charging will work between the EMG band battery and the Raspberry Pi Housing. Explain the finished EMG Band design.



(Charging cord type was determined)

Task Action Items:

Complete the Raspberry Pi housing to accommodate for the charging cord determined during the meeting.
Finalize the EMG Band design.

Meeting Minutes from March 4, 2021

Time: 1:00 – 2:30 pm

Goals:

Finalize the speaking points and the plan for the design expo, so that we can present for our TA, Ansh.

Implement any feedback from Ansh and make sure that everyone's parts flow (like a story)

Accomplishments: Decided how long we would spend on each part.

Presentation Timeline:

Total time period: 45 minutes

Questions: 20 minutes

Presentation Time: 20 minutes

Introduction + Explanation of Wearable Device: 3 minutes (max)

Modelling Team - RPi Housing Design: 8 minutes (max)

Computing Team: 8 minutes (max)

Explanation of Program: 4 minutes

Emulator Demonstration: 4 minutes

Wrap-Up/Conclusion: 1 minute

We also determined that Matthew would be responsible for the introduction, Mayar would talk about the Raspberry Pi housing. Ruidi would provide an overview of the python program, while Sharanya will explain the functions and demonstrate how the program would be implemented.

Ansh's Feedback/Notes:

- How to deal with technical difficulties
- Maybe show the EMG band design like yesterday during the presentation
- Use share screen instead of app.
- Difference in speed
- If they ask us questions one by one like Ansh, no need for transition sentences.

Task Action Items:

- Each member has to practice their own parts (pay attention to the story and flow of the presentation)
- Implement Ansh's feedback

Meeting on March 5, 2021

Time: 1:00 pm – 2:00 pm

Goal: Practice two more times before the actual design expo presentation at 2:30 pm!

Accomplishments:

Made sure that all the transitions between parts are smooth.

Determined screen-sharing responsibilities (Matthew presents the slideshow, Mayar presents the CAD model, Sharanya presents the code)

Ruidi and Sharanya ensured that each of them knows how to explain all the coding functions.

Task Action Items: Make sure we all take a break before the presentation 😊

Second Meeting on March 5, 2021

Time: 4:00 pm to 6:00 pm

Goals: Finish the team components of the Design Project Report (ie: executive summary, device design, market analysis). Make a list of responsibilities for each team member to finish by our next meeting tomorrow.

Task Action Items:

As a group:

- Finish Project Report Executive Summary (500 or less):
 - A concise summary that clearly outlines the motivation for the project and presents the design solution.
- Finish Project Report Market Analysis
 - Identifies at least 2 products on the market that are similar in scope and describe both similarities and difference.
- Finish Project Report Description of Device Design (Matthew first, then team looks over)
 - A concise description of your device including photos of solid assembly and physical components.
- Each person should add their signatures.

Individual Responsibilities:

- Manager (Ruidi):
 - Upload Preliminary Gantt Chart
 - Upload Evidence of substantial preliminary work
- Administrator (Mayar):
 - Upload all the Milestones.
 - Upload Final Gantt Chart
- Coordinator (Sharanya):
 - Upload all meeting minutes.
 - Upload Logbook
- Subject Matter Expert (Matthew):
 - Upload Source Material Database
 - Reference List (for sources used in project report)

Final Deliverables: Screenshots in the following order with title and description*(annotation):

- Matthew: wearable device (+ screenshots of prototypes)
- Mayar: rpi housing (+ screenshots of prototypes), Engineering drawings
- Ruidi: code
- Sharanya: emulator, output files, emails

*State changes made between each file, give a decent amount of description.

Meeting on March 6, 2021

Time: 1:00 pm – 3:00 pm

Goals: Finish up the Design Project Report and work on the video. Brainstorm video concepts as a group.

Task Action Items:

Mayar: Work on the appendix, engineering drawings, and preliminary work from the modelling team

Ruidi: Create a PowToon animation to create a story for the video + voiceover.

Matthew: Screen record the CAD model of the device and its design + voiceover.

Sharanya: help with editing the script and then put all the video clips together, add transitions, overlays, and text.

Meeting on March 7, 2021

Time: 1:00 pm – 2:00 pm

Goals: Final edits to Design Project Report and video.

Task Actin Items: none! We are all done 😊

Scheduled Weekly Agendas and Meeting Minutes

[January 20](#)

[January 27](#)

[February 3](#)

[February 17](#)

[February 24](#)

[March 3](#)

**Each date is hyperlinked to its corresponding agenda and meeting minutes, click to access the document on Google Drive.

Design Studio Worksheets

[Milestone 0](#)

[Milestone 1](#)

[Milestone 2](#)

[Milestone 3](#)

[Milestone 4](#)

**Each Milestone number is hyperlinked to its corresponding worksheet, click to access the document on Google Drive.

List of Sources

Source Materials Database

Milestone 1:

- [1] S.F. Chang, "Sarcopenia in the elderly: diagnosis and treatment," *Hu Li Za Zhi*, 2014 Apr; 61(2):101-5.Chinese. doi: 10.6224/JN.61.2.101. PMID: 24676957.
- [2] "Muscle Atrophy", Healthgrades, 2021. [Online]. Available: <https://www.healthgrades.com/right-care/bones-joints-and-muscles/muscle-atrophy>. [Accessed: 20- Jan- 2021].
- [3] A. Pietrangelo, "What Causes Muscle Wasting?", Healthline, Aug. 2019. [Online]. Available: <https://www.healthline.com/health/muscle-atrophy>. [Accessed: 20- Jan-2021].
- [4] S. K. Powers, G. S. Lynch, K. T. Murphy, M. B. Reid, and I. Zijdwind, "Disease -Induced Skeletal Muscle Atrophy and Fatigue," *Medicine and science in sports and exercise*, vol. 48, no. 11, pp. 2307–2319, Nov. 2016, doi: 10.1249/MSS.0000000000000975.
- [5] J. M. Argilés, N. Campos, J. M. Lopez-Pedrosa, R. Rueda, and L. Rodriguez-Mañas, "Skeletal Muscle Regulates Metabolism via Interorgan Crosstalk: Roles in Health and Disease," *J. Am. Med. Dir. Assoc.*, vol. 17, no. 9, pp. 789–796, 2016, doi: 10.1016/j.jamda.2016.04.019.
- [6] S. S. Y. Yeung et al., "Sarcopenia and its association with falls and fractures in older adults: A systematic review and meta-analysis," *J. Cachexia. Sarcopenia Muscle*, vol. 10, no. 3, pp. 485–500, 2019, doi: 10.1002/jcsm.12411.
- [7] Q. Liu, J. Gao, J. Deng, and J. Xiao, "Current Studies and Future Directions of Exercise Therapy for Muscle Atrophy Induced by Heart Failure," *Front. Cardiovasc. Med.*, vol. 7, no. October, pp. 1–6, 2020, doi: 10.3389/fcvm.2020.593429.
- [8] E. Kouidi, M. Albani, K. Natsis, et al. "The effects of exercise training on muscle atrophy in haemodialysis patients." *Nephrology, dialysis, transplantation : official publication of the European Dialysis and Transplant Association - European Renal Association*, vol. 13,3 (1998): 685-99. doi:10.1093/ndt/13.3.685

Milestone 2 (Sharanya's Individual Worksheet)

- [1] M. Ghislieri, L. Gastaldi, S. Pastorelli, S. Tadano, and V. Agostini, "Wearable inertial sensors to assess standing balance: a systematic review," *Sensors (Switzerland)*, vol. 19, no. 19, pp. 1–25, 2019, doi: 10.3390/s19194075.
- [2] M. Schwenk et al., "Wearable sensor-based in-home assessment of gait, balance, and physical activity for discrimination of frailty status: Baseline results of the Arizona frailty cohort study," *Gerontology*, vol. 61, no. 3, pp. 258–267, 2015, doi: 10.1159/000369095.
- [3] J. Pauk, A. Wasilewska, and M. Ihnatouski, "Infrared thermography sensor for disease activity detection in Rheumatoid arthritis patients," *Sensors (Switzerland)*, vol. 19, no. 16, pp. 1–14, 2019, doi: 10.3390/s19163444.
- [4] "NSAIDs: Arthritis Foundation," *NSAIDs | Arthritis Foundation*. [Online]. Available: <https://www.arthritis.org/drug-guide/nsaids/nsaids#:~:text=Nonsteroidal%20anti%2Dinflammatory%20drugs%20>. [Accessed: 06-Mar-2021].
- [5] W. Hardin, "Thermal Imaging to Diagnose Disease," *Vision Online*, 14-Aug-2018. [Online]. Available: https://www.visiononline.org/vision-resources-details.cfm/vision-resources/Thermal-Imaging-to-Diagnose-Disease/content_id/6938. [Accessed: 06-Mar-2021].

Final Deliverable:

- [1] Humberto Rocha, "Sending and receiving emails with Python," *Humberto Rocha*, 29-Jul-2020. [Online]. Available: <https://humberto.io/blog/sending-and-receiving-emails-with-python/>. [Accessed: 06-Mar-2021].

References

- [1] J. D. Walston, “Sarcopenia in older adults,” *Curr. Opin. Rheumatol.*, vol. 24, no. 6, pp. 623–627, 2012, doi: 10.1097/BOR.0b013e328358d59b.
- [2] S. von Haehling, J. E. Morley, and S. D. Anker, “An overview of sarcopenia: Facts and numbers on prevalence and clinical impact,” *J. Cachexia. Sarcopenia Muscle*, vol. 1, no. 2, pp. 129–133, 2010, doi: 10.1007/s13539-010-0014-2.
- [3] S. Z. Yoo *et al.*, “Role of exercise in age-related sarcopenia,” *J. Exerc. Rehabil.*, vol. 14, no. 4, pp. 551–558, 2018, doi: 10.12965/jer.1836268.134.
- [4] “Memory, Forgetfulness, and Aging: What's Normal and What's Not?,” *National Institute on Aging*. [Online]. Available: <https://www.nia.nih.gov/health/memory-forgetfulness-and-aging-whats-normal-and-whats-not>. [Accessed: 06-Mar-2021].
- [5] “Mshorts 3,” *Myontec*. [Online]. Available: <https://www.myontec.com/product-page/mshorts3>. [Accessed: 06-Mar-2021].
- [6] “MCell 3,” *Myontec*. [Online]. Available: <https://www.myontec.com/product-page/mcell-3>. [Accessed: 06-Mar-2021].
- [7] “Mbody 3 Kit Legs,” *Myontec*. [Online]. Available: <https://www.myontec.com/product-page/mbody-3>. [Accessed: 06-Mar-2021].
- [8] “Shimmer3 EMG Unit,” *Shimmer*. [Online]. Available: <https://www.shimmersensing.com/products/shimmer3-emg-sensor#features-tab>. [Accessed: 06-Mar-2021].
- [9] G. Halfacree, “Arduino Announces Raspberry Pi RP2040 Core Port, Arduino Nano RP2040 Connect Board,” *Hackster.io*, Feb-2021. [Online]. Available: <https://www.hackster.io/news/arduino-announces-raspberry-pi-rp2040-core-port-arduino-nano-rp2040-connect-board-615085ce4791>. [Accessed: 06-Mar-2021].
- [10] “RAVPower 2800mAh Galaxy S4 Li-ion Replacement Battery with NFC Function for Samsung I337, Verizon I545, Sprint L720, T-Mobile M919, R970, I9500, I9505 (Not for Galaxy S4 Active or S4 Mini),” *Amazon.ca: Cell Phones & Accessories*. [Online]. Available: <https://www.amazon.ca/RAVPower-2800mAh-Replacement-Function-T-Mobile/dp/B00CR6NQT2>. [Accessed: 06-Mar-2021].
- [11] “Guide to Understanding Activewear Fabrics,” *Surge Fabric Shop*. [Online]. Available: <https://surgefabricshop.com/blogs/surges-sewing-blog/guide-to-understanding-activewear-fabrics>. [Accessed: 06-Mar-2021].

Appendices

Appendix A: Autodesk CAD Modelling

A1. Raspberry Pi Housing Design:

A1.1: the full assembly of the pi housing:

Multi View:



Figure 1 A1.1: From this view we can see the Logo and the brand name being extruded as holes to allow for better venting and to reduce material used in 3D printing.

Side view:

Figure 2 A1.1.

Front view:

Figure 3 A1.1: Additional air vents on the sides of the housing to have a symmetry in the air flow and reduce the pressure on the front and back vents.

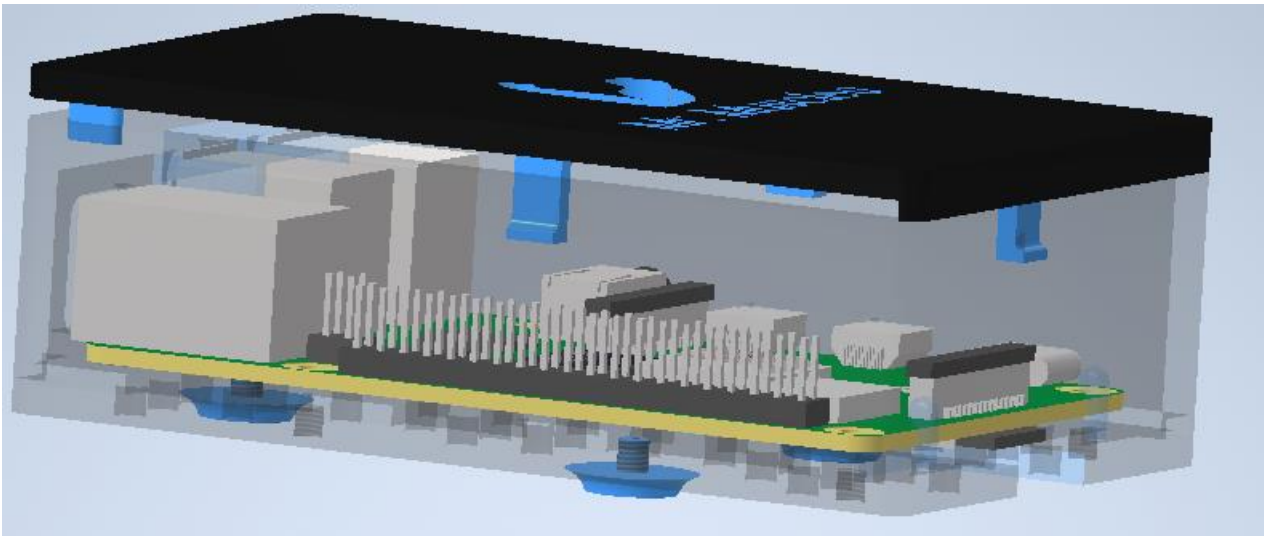
Transparent housing Multiview:

Figure 4 A1.1: We can see the cantilever snap-fits of the lid holding the cylinders on the inside of the housing, we can also see the bottom air vents and the feet screwed in their place in the housing.

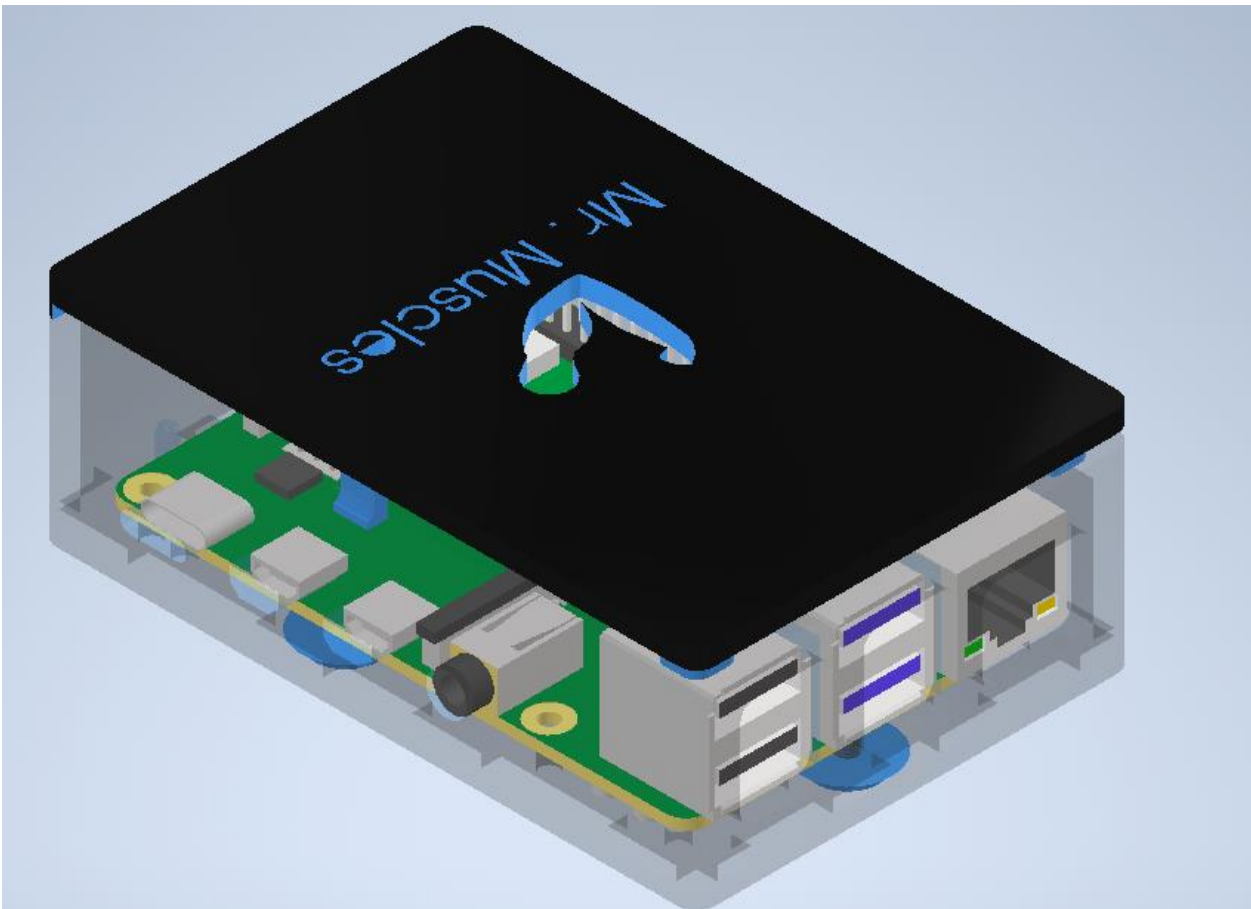


Figure 5 A1.1: charging the EMG band will be completed through a charging cord connected to one USB ports of the pi. Thus, charging docks problem were resolved.

A1.2 Pi Housing:

Multi view:

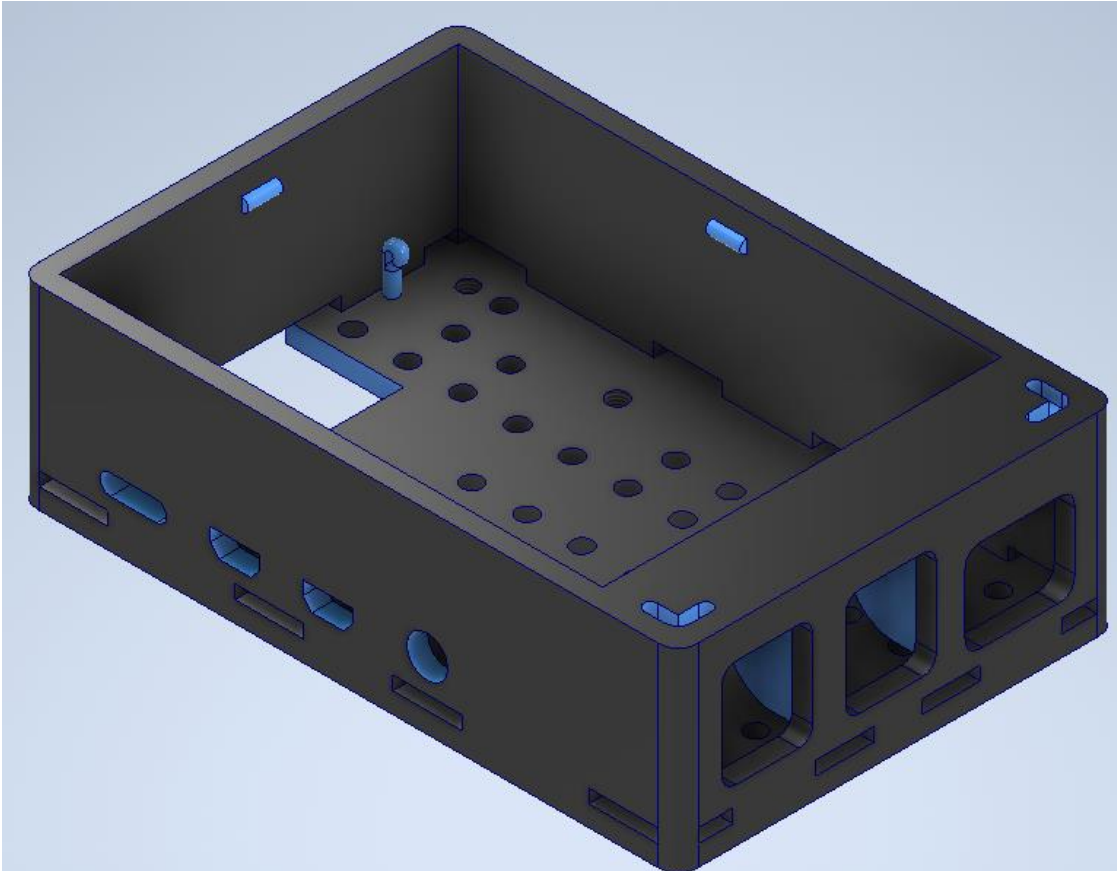


Figure 1 A1.2.

Front view:



Figure 2 A1.2.

Top view:

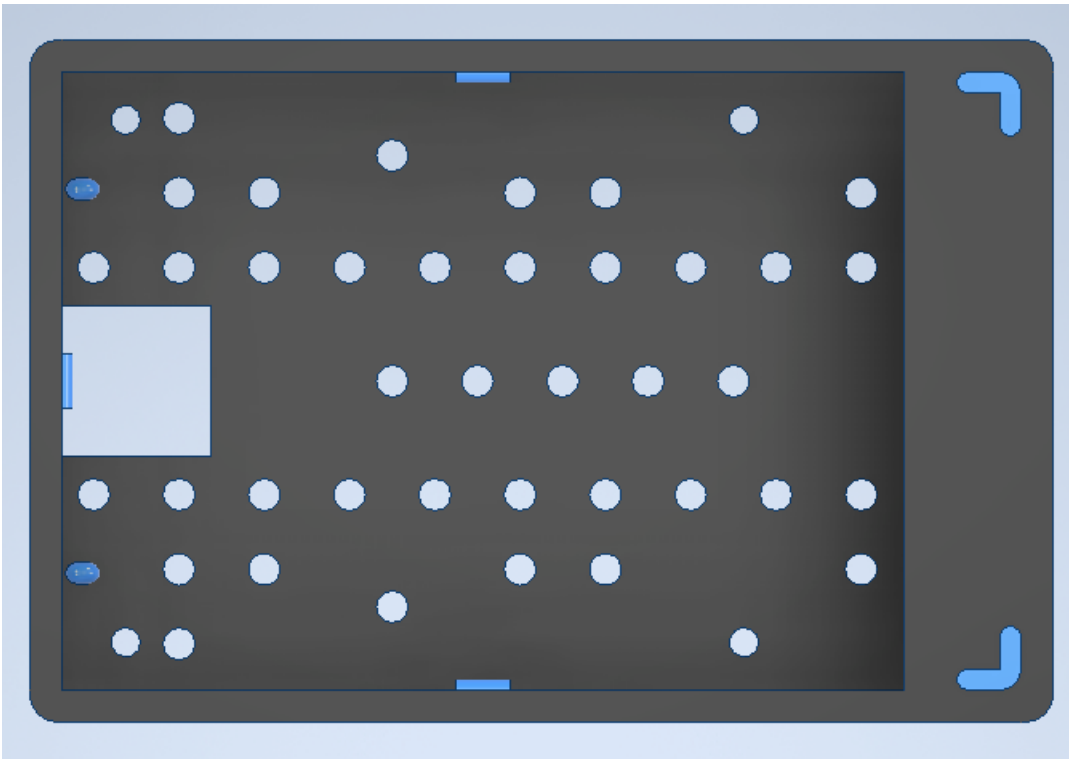


Figure 3 A1.2: The pattern of the circles at the bottom is for air venting, the L shape and the cylindrical extrusions are for supporting the lid.

Left view:



Figure 4 A1.2: The gap for the micro-SD card is highlighted in blue.

Back view:

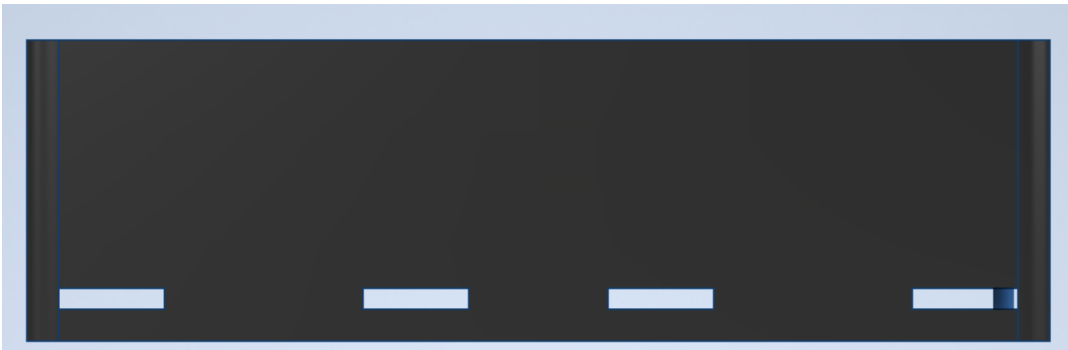


Figure 5 A1.2.

Right view:

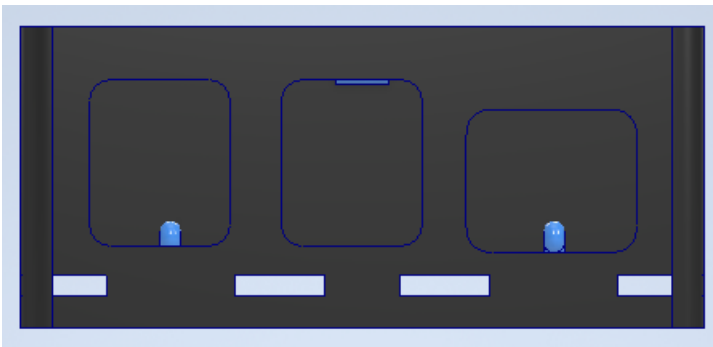


Figure 6 A1.2.

Bottom view:

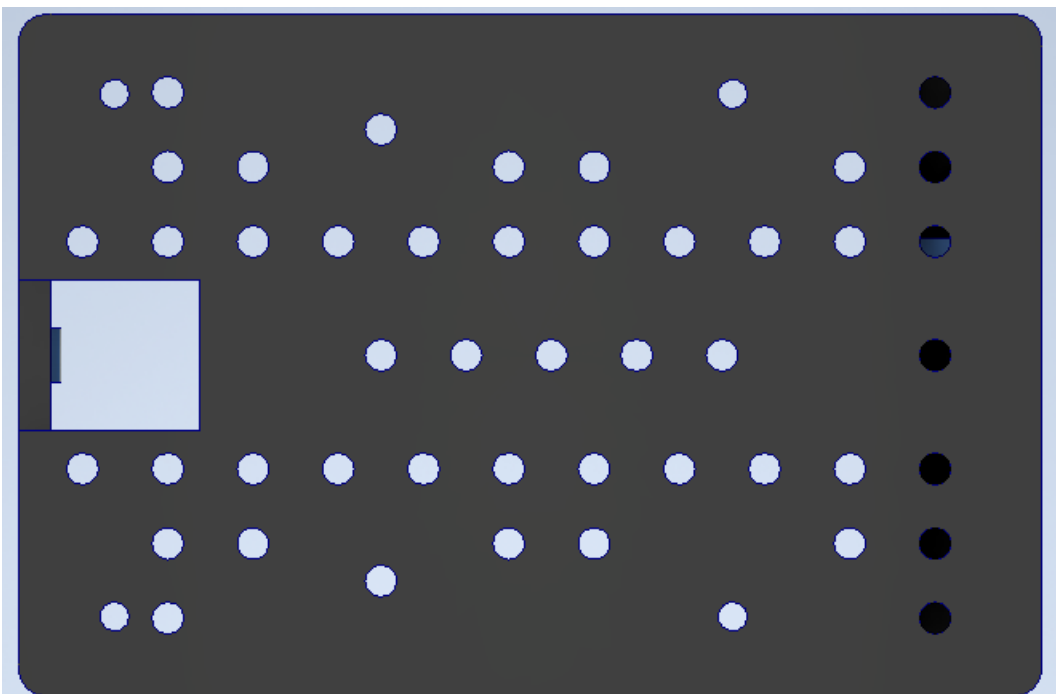


Figure 7 A1.2: The pin at the bottom will let the pi chip snap in its place and prevent its movement in the housing, the dividers (seen in blue through the USB ports) are also holding the pi chip from the other side which will work with the spherical pins to prevent the movement and rotation of the chip.

The L shape and cylindrical shapes are explained in the previous picture and they can support the lid and keep it in place using snap-fit mechanism.

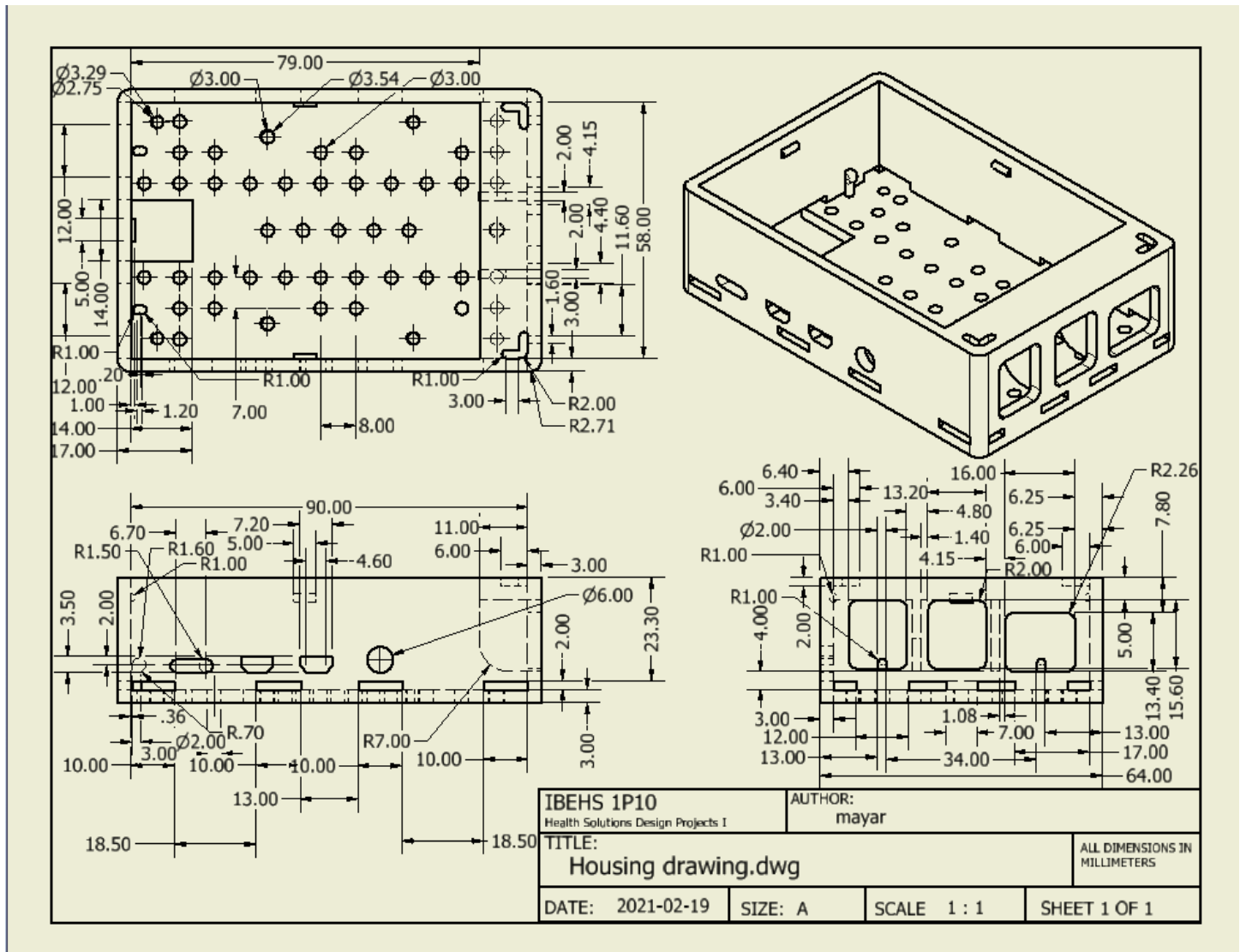


Figure 8 A1.2: The engineering drawing of the Housing.

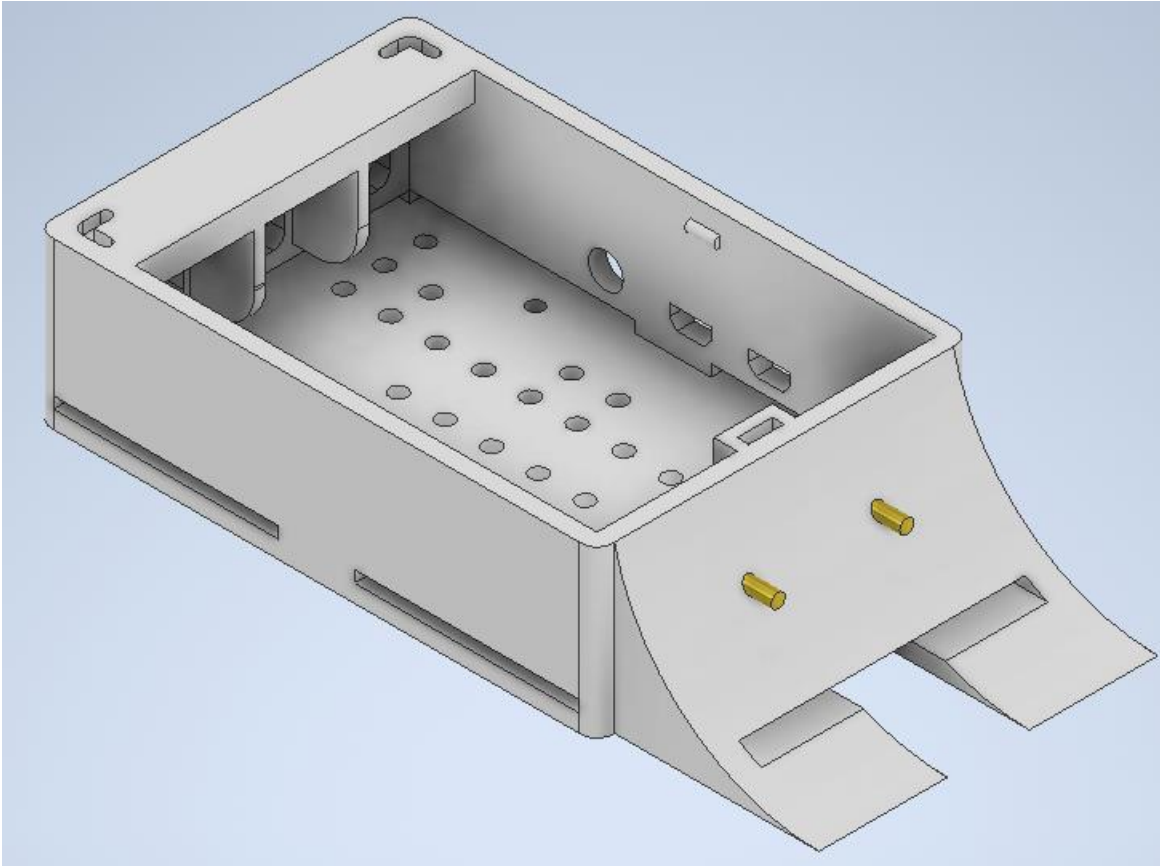
A1.2.2 Housing prototypes:**1st prototype:**

Figure 1 A1.2.2: This prototype was rejected because of the arc shape made by the charging dock. Charging docks designs were rejected in general because we decided that it is better to charge the EMG band with the charging cord connected to one USB port. The holed square in front of the charging dock is also changed to become a cylindrical shape like the one on the sides of the housing, this was applied to reduce support material and have a more symmetrical shape.

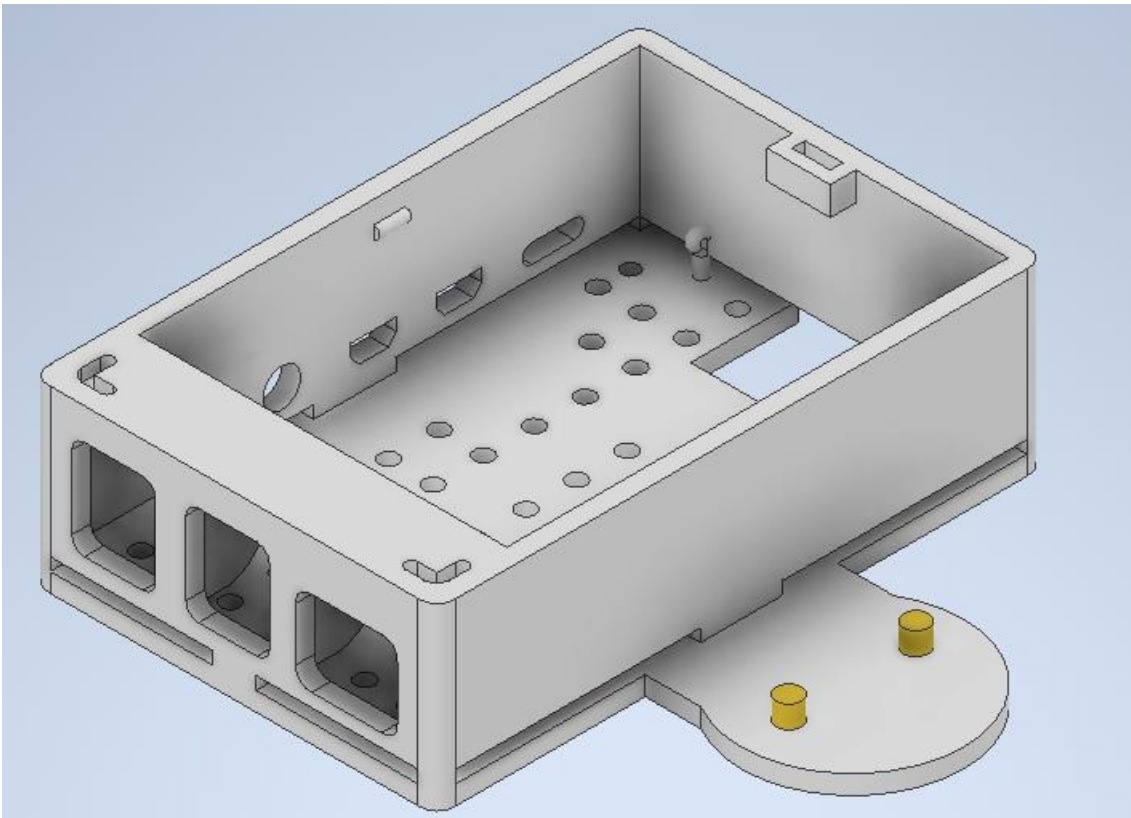
2nd prototype:

Figure 2 A1.2.2: All charging docks were removed because we settled with the idea of connecting a charging cord to the USB port.

3rd prototype:

Figure 3 A1.2.2: This model was rejected because our customer demographics target elderly people suffering of muscle atrophy. It would have been perfect if the target customers were children with muscle atrophy. Our brand name “Mr. Muscles” was inspired by this design.

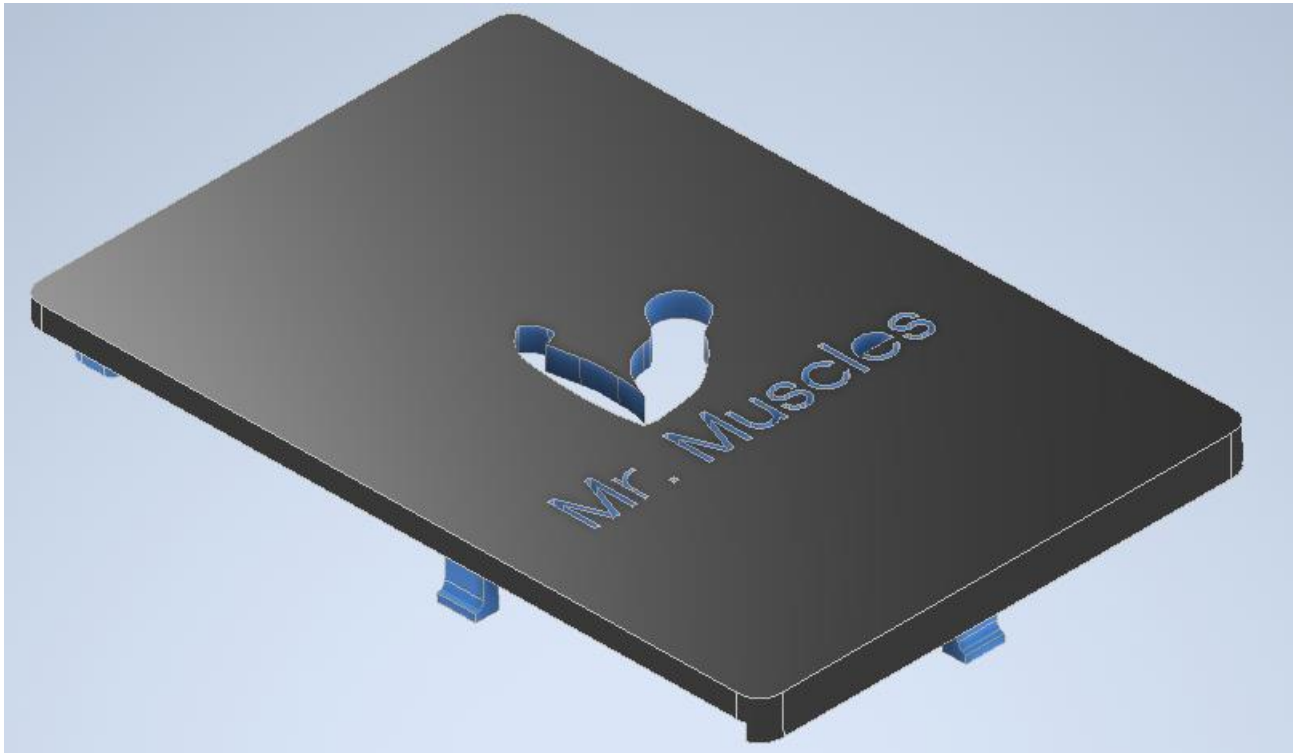
A1.3 the lid**Multi view 1:**

Figure 1 A1.3.

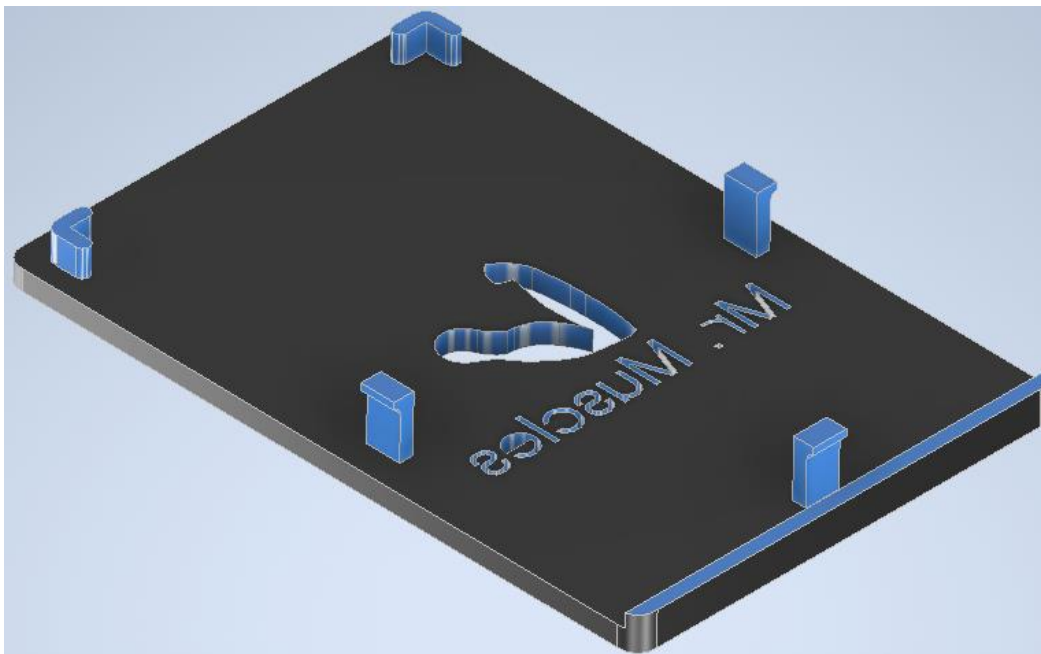
Multi view 2:

Figure 2 A1.3.

Top view:

Figure 3 A1.3: The Logo and the brand name being extruded as holes to allow for better venting.

Front view:

Figure 4 A1.3: The L shape extrusions are seen on the left of the picture and the cantilever snap-fits are on the middle and right of the picture.

Right view:

Figure 5 A1.3.

Bottom view:

Figure 6 A1.3. The three-cantilever snap-fits (looking like three squares on the side and bottom) are to support the lid and prevent it from falling off the housing or being removed by applying minute force. The edged rectangle and the L shapes at the top and bottom of the lid are supporting the lid from falling on the pi housing.

Back view:

Figure 7 A1.3.

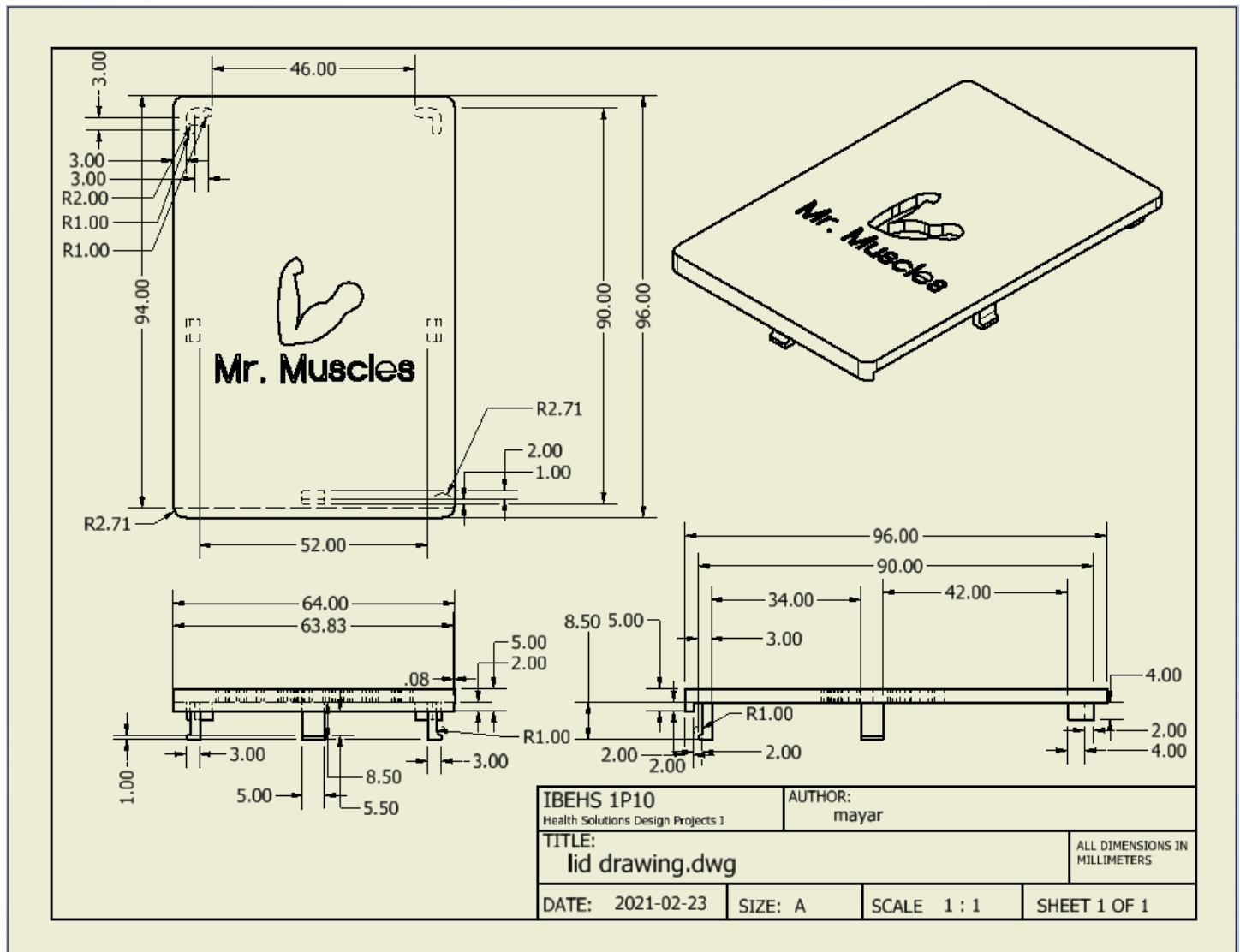


Figure 8 A1.3: The engineering drawing of the Lid

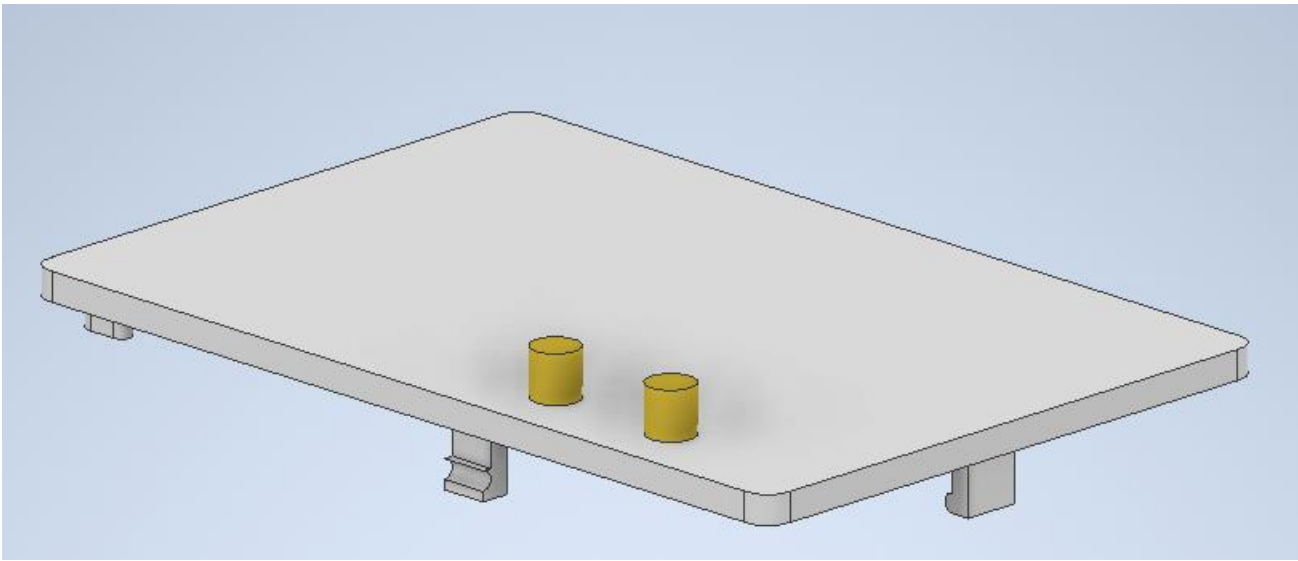
A1.3.2 the lid prototype:

Figure 1 A1.3.2: The top portion of the sphere of the cantilevers were removed because of its thinness which would make it prone to breaking, the cantilever on the right side of the image (front side of the model) is also changed to match the other two on the left and right side of the model. The charging port in the top of the lid was removed because having it will need a lot of supporting material while 3D printing and because we resolved the charging dock problem.

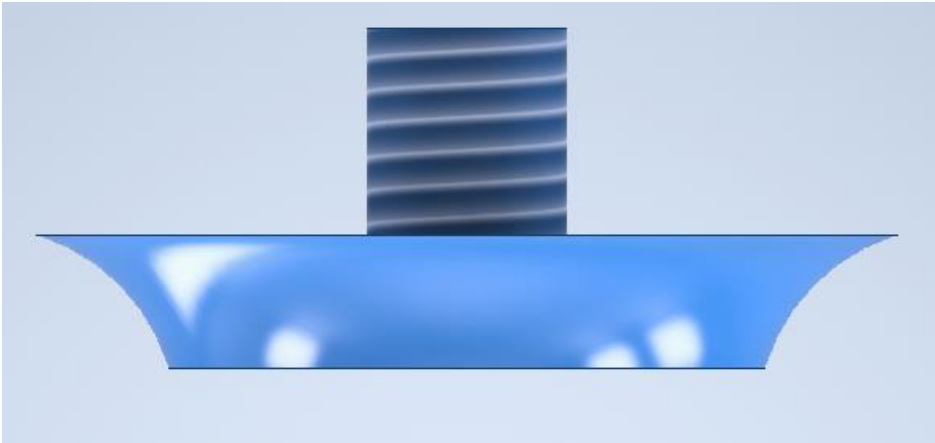
A1.4 the feet:**Front view:**

Figure 1 A1.4: The feet are necessary to provide a gap between the bottom venting holes in the housing and the surface that the housing would settle on, the gap is necessary for the mentioned hole to function and provide flow. The feet are being printed individually to reduce the support material needed for the printing of the housing.

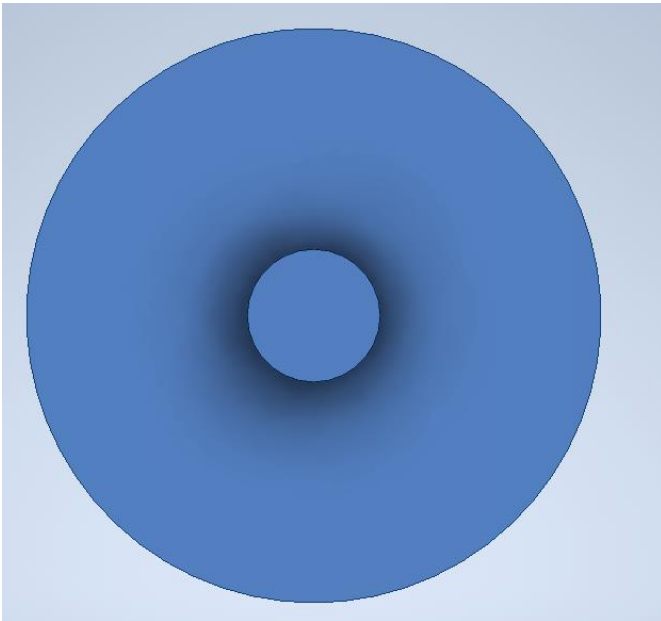
Top view:

Figure 2 A1.4.

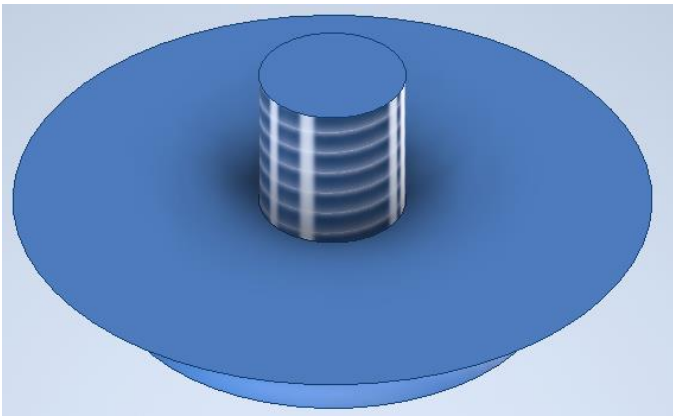
Multiview:

Figure 3 A1.4.

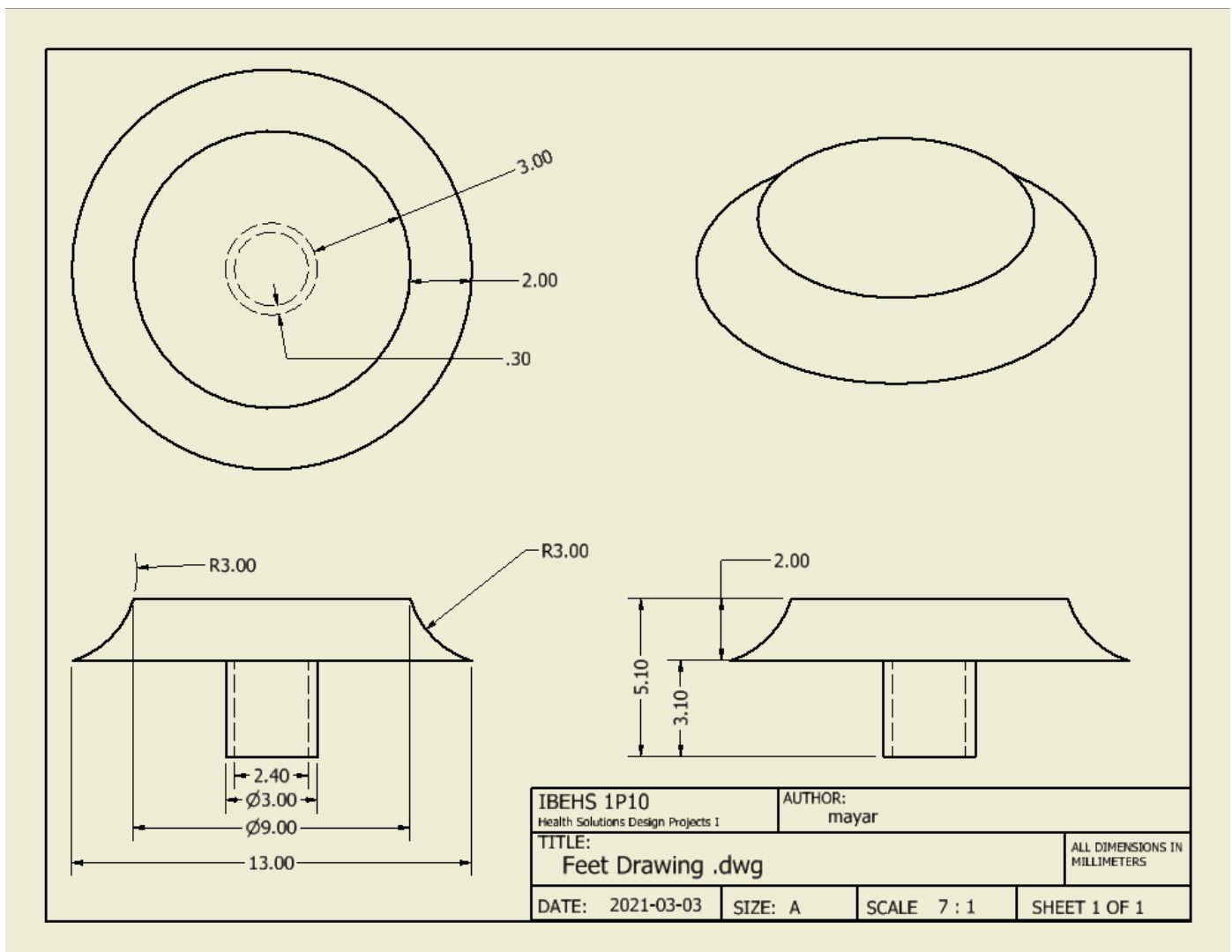


Figure 4 A1.4: The engineering drawing for the feet

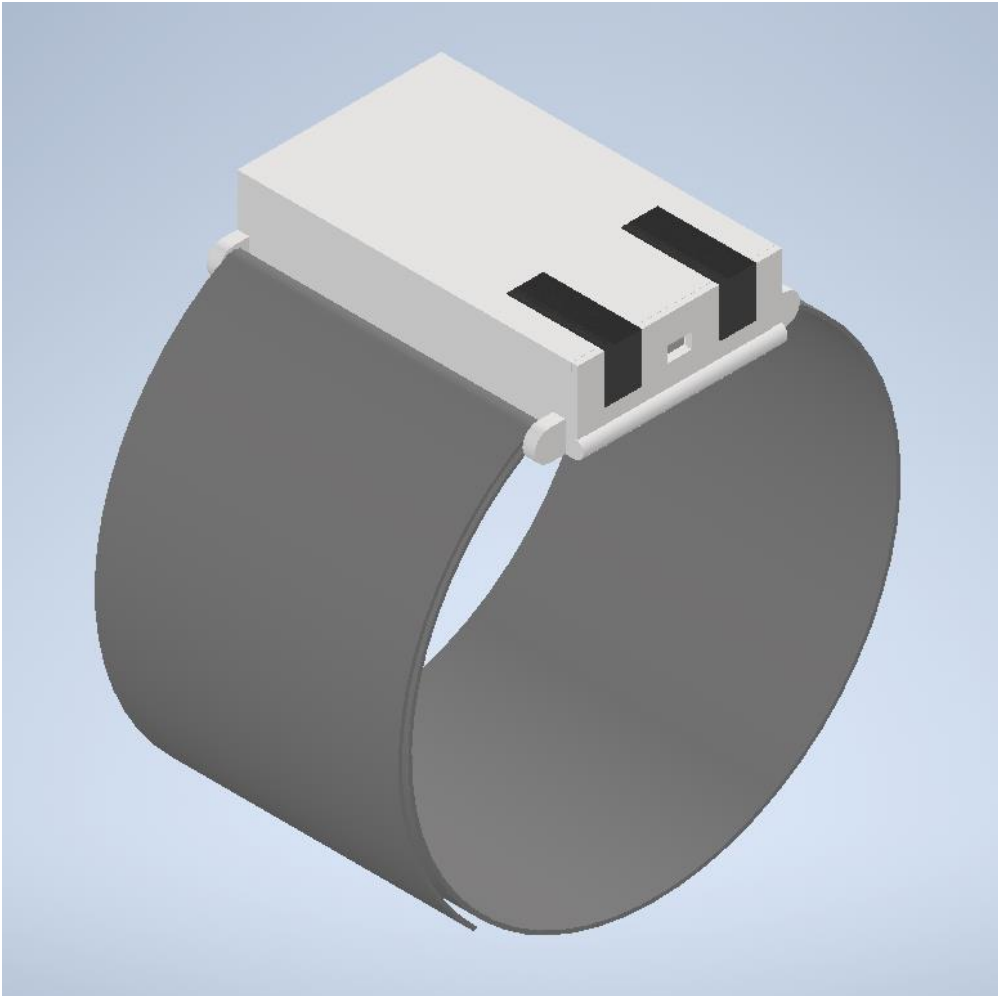
A2. Device Design (Mr. Muscles EMG Band):**Multiview:**

Figure 1.A2: Outer view of the Mr. Muscles EMG Band, with the lid closed and both Velcro straps closed over top of the lid. The lid is intended to stay closed during client usage and charging and should only be opened for maintenance and repair. Charging the battery is accessible via the charging port hole located on the lid.

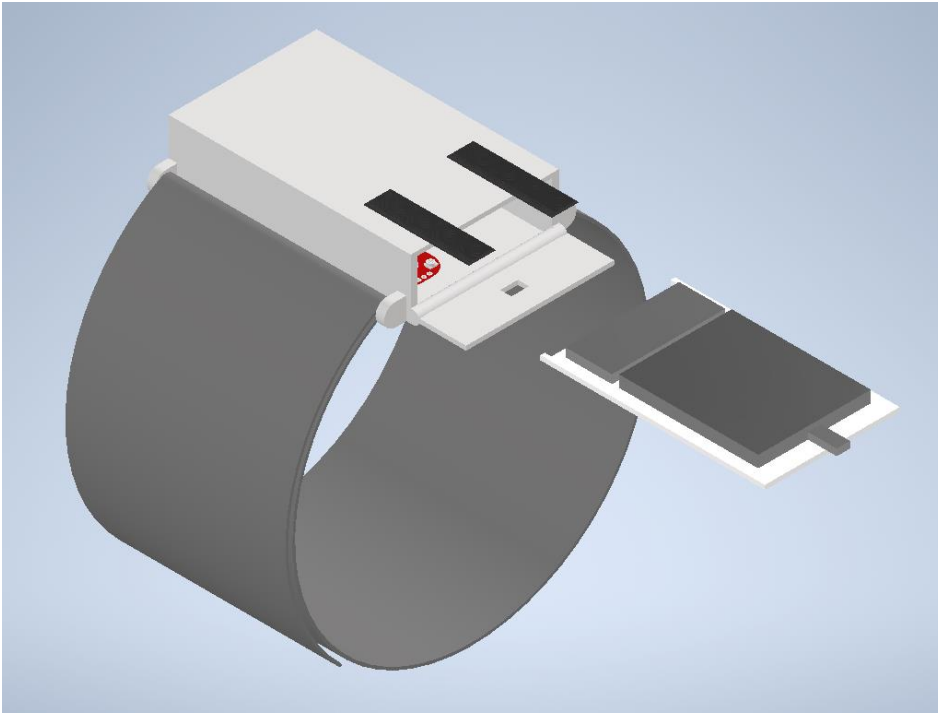


Figure 2.A2: Outer view of the Mr. Muscles EMG Band, with both Velcro straps opened over top of the lid and the lid opened. The second layer has been pulled out of the casing to model what the removed layer looks like.

Front View:

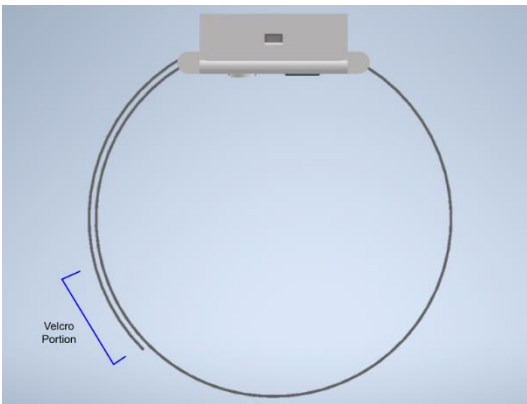


Figure 3.A2: Front view of the Mr. Muscles EMG Band, with the lid closed and both Velcro straps closed over top of the lid. The charging hole aligns directly with the charging port connected to the battery inside. The end portion of the Spandex band has Velcro that allows the band to remain tightened around the thigh.

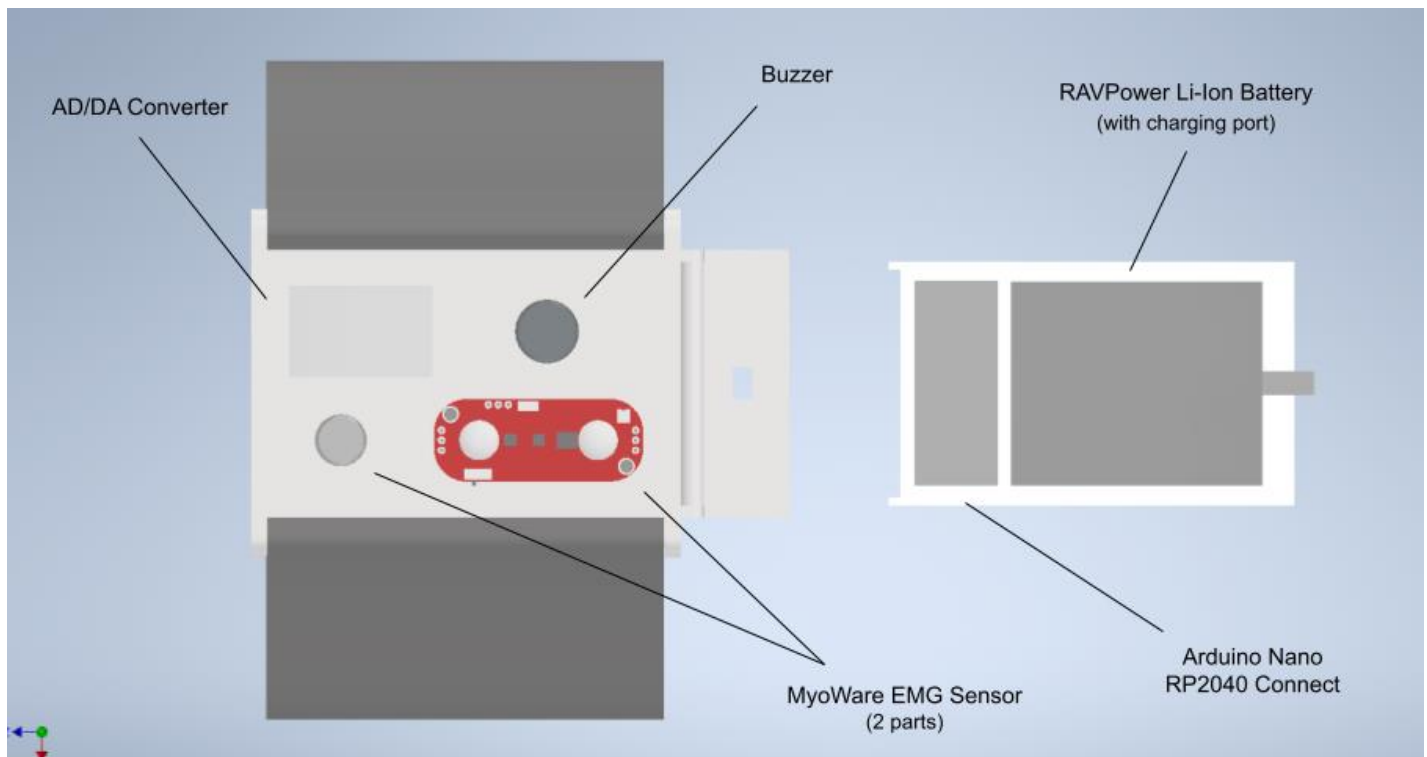
Top View:

Figure 4.A2: Top view of the Mr. Muscles EMG Band, with the lid opened, Velcro straps and top layer of the casing made transparent for better visibility, and the second layer removed from the casing. The bottom layer consists of the MyoWare EMG Sensor (2 parts), buzzer, and AD/DA converter. The top layer consists of the Arduino Nano RP2040 Connect and the RAVPower 2800mAh Li-Ion batter + charging port. At the rear end of the second layer, a gap is included to allow for wires to pass between the layers.

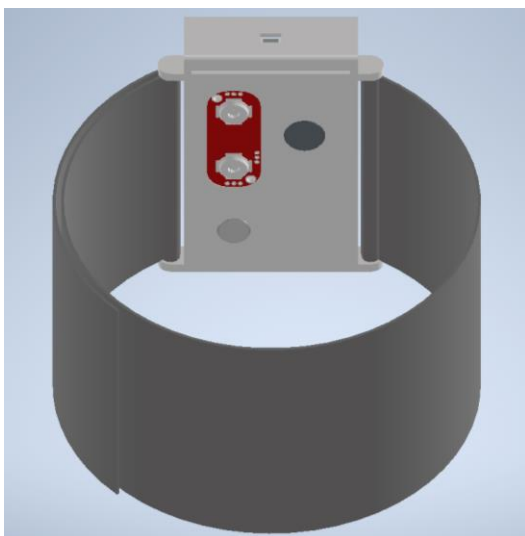
Underside (Bottom) View:

Figure 5.A2: Underside (bottom) view of the Mr. Muscles EMG Band. Both components of the MyoWare EMG Sensor and the buzzer protrude through the casing and come into direct contact with the skin.

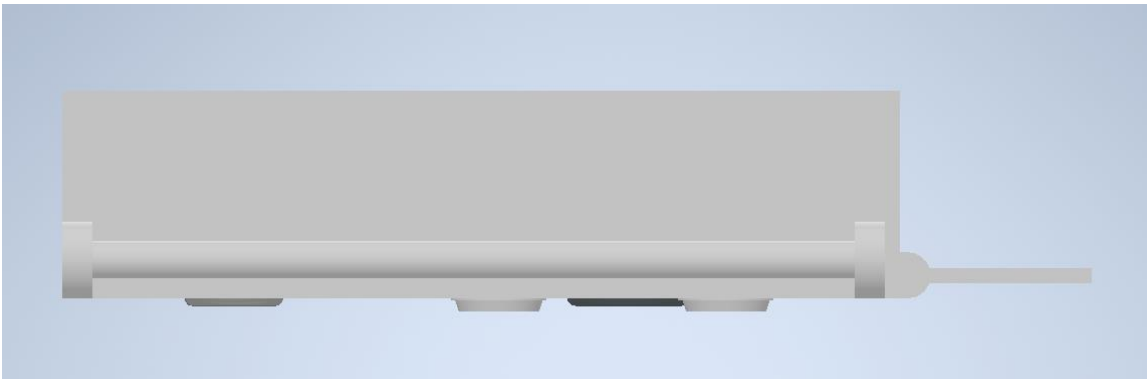
Side View

Figure 6.A2: Side view of the Mr. Muscles EMG Band. The lid is left open, and the MyoWare EMG sensor and buzzer protrusions are visible on the bottom of the casing. The cylindrical lug on the side of the casing is what the Spandex elastic band is wrapped around on either side

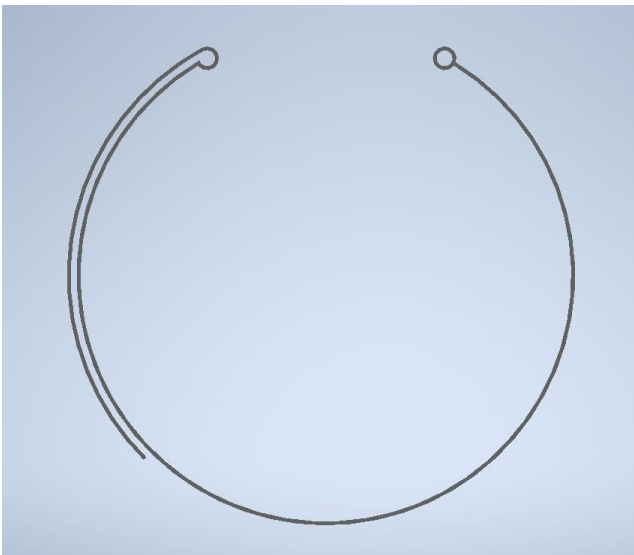
Band View:

Figure 7.A2: Front view of the Mr. Muscles EMG Band, with only the Spandex elastic band visible. The right side is permanently attached to the cylindrical lug on the casing and cannot be shortened/lengthened. The left side is inserted underneath the cylindrical lug and is wrapped around the top side, ultimately creating a mechanism that tightens the whole band if the loose end is pulled and loosens the whole band if the loose end is shortened.

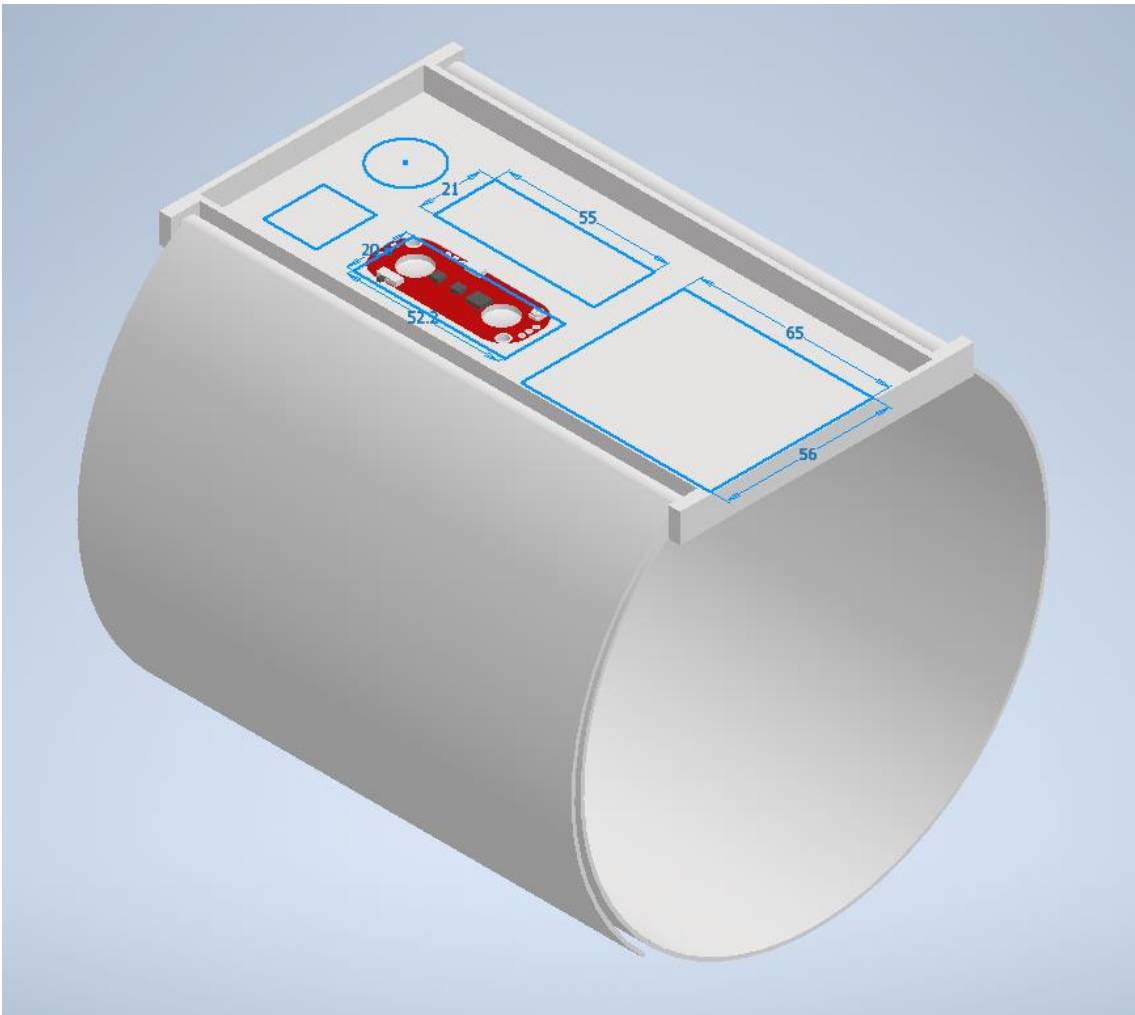
Prototype View:

Figure 7.A2: Multiview of the Mr. Muscles EMG Band prototype. Only one prototype was devised before the final version was created. The rectangles in the 2D sketch illustrate the potential layout of the various components located within the EMG Band casing. Using only 1 layer, the length of this casing became inconveniently long for the user to wear on their thighs, so moving forwards the final design needed a shortened length. The same concept for the Spandex band is used, where cylindrical lugs are located on either side of the casing and the band interacts with them in an identical manner as the final version. This version is also missing the second component of the MyoWare EMG sensor, as well as protrusions in the base layer to gain access to the skin, and therefore must be added into the final version.

Appendix B: Computer Program

```
## Write your code below this line

def calc_rolling_average():
    while i < len(data_list) - average_number + 1:
        data_set = data_list[i : i + average_number]
        set_average = sum(data_set) / average_number
        rolling_average.append(set_average)
        i += 1
    return(rolling_average)

def calc_time_rest():
    for i in rolling_averages:
        if i <= upper_resting_EMG:
            "Find Code for Start time"
            while time_period_rest > prescribed_time_interval:
                buzzer.on(30) # add the time for the buzzer
                if i > upper_resting_EMG:
                    buzzer.off()
            else:
                buzzer.on(30) # add the time for the buzzer
                buzzer.off()
                time.sleep(120)
                buzzer.on() # add the time for the buzzer
                buzzer.off()
                time.sleep(600)
                buzzer.on() # add the time for the buzzer
                buzzer.off()
                time.sleep(1800)

def output_device #(message user)
...

*****
***  Function Definitions - STUDENT CODE ENDS HERE!  ***
```

Figure 1B: Screen shot of computer program file 1.0.

This is the first python file written for the personal wearable design. It includes two preliminary functions: `calc_rolling_average` and `calc_time_rest`.

```
## Start with indentation aligned with this comment
import time
sensor = Muscle_Sensor()
buzzer = Buzzer (18)
rolling_average = []
data_list = []

while True:
    data_point = sensor.muscle_scaled(10)
    print("Raw Data:", data_point)
    data_list.append(data_point)
    rolling_average = calc_rolling_average(data_list)

...

*****
***  Program Implementation - STUDENT CODE ENDS HERE!  ***
*****
```

Figure 2B: Screen shot of implementation section of computer program file 1.5.

This file has created an instance for both the muscle sensor class and the buzzer class to file 1.0 . A while loop was created to continuously print and append new data to the initialized list named data_list from the EMG sensor.

```

...
*****
*** Function Definitions - STUDENT CODE BEGINS HERE! ***
*****
Below this line is where you should DEFINE all functions
- Any output devices that are used in your functions
  should be defined here as well
- PAY ATTENTION TO YOUR INDENTATIONS!!!!
...
## Write your code below this line

def calc_rolling_average(data_list):
    n_items = 5
    if (len(data_list) >= n_items):
        average_list = []
        for i in range(1, (n_items + 1)):
            average_list.append(data_list[-i])
        rolling_average = round(sum(average_list)/(n_items),2)
        ## print ('Rolling Average:', rolling_average)
        return rolling_average
    else:
        None

def calc_rest(rolling_average):
    prescribed_time_interval = 20*60
    active_EMG = 50
    upper_resting_EMG = 0
    for i in rolling_average:
        if i <= active_EMG:
            ## start_time = time.time()
            ## current_time = time.time()
            ## elapsed_time = current_time - start_time
            ## print(elapsed_time)
            while elapsed_time > prescribed_time_interval:
                buzz_text_reminder(i)
            else:
                break

```

Figure 3B: Screen shot of function definitions of computer program file 2.0 (Part 1)

The calc_rolling_average function was improved based on the function written in file 1.0. A section of the calc_rest function is commented out because we were not sure how to write a program that act as a timer.

```

##put all buzzer reminders into ONE FUNCTION, call that function whenever we want the buzzers to go off
def buzz_text_reminder(i):
    buzzer.on()
    time.sleep(30) #duration of sound
    print("It is time to exercise again.")
    if i >= upper_resting_EMG:
        buzzer.off()
    else:
        buzzer.on()
        time.sleep(30) #duration of sound
        print("It is time to exercise again.")
        buzzer.off()
        time.sleep(120) #time before next reminder
        buzzer.on()
        time.sleep(30) #duration of sound
        print("It is time to exercise again.")
        if i >= upper_resting_EMG:
            buzzer.off()
        else:
            buzzer.off()
            time.sleep(600) #time before next reminder
            buzzer.on()
            time.sleep(30) #duration of sound
            print("It is time to exercise again.")
            buzzer.off()
            time.sleep(600) #time before next reminder
        while i <= upper_resting_EMG:
            buzzer.on()
            time.sleep(30) #duration of sound
            print("It is time to exercise again.")
            buzzer.off()
            time.sleep(600) #time before next reminder
        else:
            buzzer.off()

def file(file_name,data_list,data_point,rolling_average):
    file = open(file_name, 'a')
    if len(data_list)== 1:
        open(file_name, 'w').close()##this will clear the File's contents
        file.write("EMG(Raw)" + '\t' + "EMG(Avg)" + '\n' + str(data_point) + '\n')
        file.close()
    else:
        file.write(str(data_point) + '\t' + str(rolling_average) + '\n')
        file.close()

```

Figure 4B: Screen shot of function definitions of computer program file 2.0. (Part 2)

2 functions were created in file 2.0: the buzz_text_reminder function and the file function. The buzz_text_reminder here was planned to remind the user after 30 seconds they have stopped exercising, another reminder will be made after 2mins and 10 mins. The sound duration of the buzzer is 30s and if they exercise again the reminder will turn off. The file function will output raw data points and rolling averages as a text file.

```
try:
    """
    ****
    *** Program Implementation - STUDENT CODE BEGINS HERE! ***
    ****
    Below this line is where you should IMPLEMENT each of the
    required tasks.
    - DO NOT define any functions in this section!
    - You can liken this to a main() function where you
      would implement your individual functions in your
      Lab Mini-Milestones
    - PAY ATTENTION TO YOUR INDENTATIONS!!!!
    """
    ## Start with indentation aligned with this comment
    import time
    sensor = Muscle_Sensor()
    buzzer = Buzzer (18)
    rolling_average = []
    data_list = []

    while True:
        data_point = sensor.muscle_scaled(10)
        ## print("Raw Data:", data_point)
        data_list.append(data_point)
        rolling_average = calc_rolling_average(data_list)
        ## print(data_list)
        file('testingfile.txt',data_list,data_point,rolling_average)
        calc_rest(rolling_average)

    """
    ****
    *** Program Implementation - STUDENT CODE ENDS HERE! ***
    ****

except:
    (type, value, traceback) = sys.exc_info()
    sys.excepthook(type, value, traceback)
    sys.exit(1)
def stop(self):
    self.is_running = False
    print('stopping thread...')
    self.terminate()
```

Figure 5B: Screen shot of implementation section of computer program file 2.0.

The file function was added and called in the implementation section.

```
def calc_rest(rolling_average):
    prescribed_time_interval = 20*60
    active_EMG = 50
    upper_resting_EMG = 0
    for i in rolling_average:
        if i <= active_EMG:
            ## start_time = time.time()
            ## current_time = time.time()
            ## elapsed_time = current_time - start_time
            ## print(elapsed_time)
            while elapsed_time > prescribed_time_interval:
                buzz_text_reminder(i)
            else:
                break
```

Figure 6B: Screen shot of calc_rest function of computer program file 3.0.

With the help from Bola, Parmveer, the tech assistance of Design Project 3, we improved our idea for calc_rest function by using the time method(time.time()).

```
def calc_rest(rolling_average):
    import time
    active_EMG = 4
    time_interval = 10
    rest_time = 0
    sound_duration = 30
    for i in rolling_average:
        print("voltage:", i)
        if i >= active_EMG:
            rest_time = 0
            print(rest_time)
        if i < active_EMG:
            time.sleep(1)
            rest_time += 1
            print(rest_time)
        while rest_time > time_interval:
            buzzer.on()
            time.sleep(sound_duration) #duration of sound
            print("It is time to exercise again.")
            if i >= active_EMG:
                buzzer.off()
            else:
                while i <= active_EMG: #Maybe change it to while True
                    buzzer.on()
                    time.sleep(sound_duration) #duration of sound
                    print("It is time to exercise again.")
                    buzzer.off()
```

Figure 6B: Screen shot of calc_rest function of computer program file 4.0.

The idea of using the time method was abandoned because it does not fit well with our objective of this function and it is hard to use. We have decided to go back to nested loops and improved the calc_rest function. The rest_time was originally set to zero, if the patient's voltage is less than active EMG, the rest_time variable adds one for every iteration until it is larger than the time_interval. Once it is greater than the time_interval, the buzzer turns on. The calc_rest function in the files after 4.0 version are improved based on this method of calculating the patient's rest time.

```

## Write your code below this line
#This function is used to calculate the rolling average and outputting as a single data (not a list)

def calc_rolling_average(data_list):
    n_items = 10 #the number of items needed for calculating one average, this value could be changed
    if (len(data_list) >= n_items): #once the number of data points exceeds items, rolling average can be made
        average_list = [] #initializing the list of average numbers
        for i in range(1, (n_items + 1)): #range of values is equal to items
            average_list.append(data_list[-i]) #adds the previous number to the list
        rolling_average = round(sum(average_list)/(n_items),2) #calculates the average, rounded to two decimal places
        return rolling_average #returns the value of the rolling average

#This function is used to calculate the user's rest time, the time interval when they are not meeting the exercise expectations

def calc_rest(rolling_average):
    active_EMG = 4 #This value is used for demonstration, usually its given to the patient from the physician
    rest_time = 0
    time_interval = 10 #This value is used for demonstration, in reality, the resting time interval will be much longer (e.g. 30mins)
    print("voltage:", rolling_average)
    if rolling_average != None and rolling_average >= active_EMG:
        rest_time = 0
    else:
        while rolling_average != None and rolling_average < active_EMG:
            rest_time += 1
            if rest_time > time_interval:
                break #Allows the code to move on in the main function once the user's resting time exceeds the prescribed resting interval
        return rest_time

#This functions reminds the user with both a buzzer and text message
#Once the user exceeds the resting time_interval, the buzzer turns on and sends a text message to the user's phone(in this case the shell)
#If the user start exercising again, the buzzer will turn off
def buzz_text_reminder(rest_time,rolling_average):
    buzzer = Buzzer(18)
    sound_duration = 5 #This value could be changed by the user
    time_interval = 10 #This value is used for demonstration, in reality, the resting time interval will be much longer (e.g. 30mins)
    active_EMG = 4 #This value is used for demonstration, usually its given to the patient from the physician
    if rest_time > time_interval:
        buzzer.on()
        time.sleep(sound_duration) #duration of sound
        print("It is time to exercise again.") #The text message is sent to the user's phone
        if rolling_average != None and rolling_average >= active_EMG:
            buzzer.off()
        elif rolling_average != None and rolling_average <= active_EMG: #Maybe change it to while True
            buzzer.on()
            time.sleep(sound_duration) #duration of buzz sound
            print("It is time to exercise again.") #The text message is sent to the user's phone
            buzzer.off()

```

Figure 7B: Screenshot of function definitions of computer program file 5.0 (Part 1).

Comments were added to the file to explain the functions. This file became the template of future files. The `calc_rest` function was refined to avoid comparing `None` type variables to finite values. A `break` statement is added to the end of the `while` loop in the function which allows the codes to move on in the implementing section of the program.

```

#This functions reminds the user with both a buzzer and text message
#Once the user exceeds the resting time_interval, the buzzer turns on and sends a text message to the user's phone(in this case the shell)
#If the user start exercising again, the buzzer will turn off
def buzz_text_reminder(rest_time,rolling_average):
    buzzer = Buzzer(18)
    sound_duration = 5 #This value could be changed by the user
    time_interval = 10 #This value is used for demonstration, in reality, the resting time interval will be much longer (e.g. 30mins)
    active_EMG = 4 #This value is used for demonstration, usually its given to the patient from the physician
    if rest_time > time_interval:
        buzzer.on()
        time.sleep(sound_duration) #duration of sound
        print("It is time to exercise again.") #The text message is sent to the user's phone
        if rolling_average != None and rolling_average >= active_EMG:
            buzzer.off()
        elif rolling_average != None and rolling_average <= active_EMG: #Maybe change it to while True
            buzzer.on()
            time.sleep(sound_duration) #duration of buzz sound
            print("It is time to exercise again.") #The text message is sent to the user's phone
            buzzer.off()

#This function outputs the input datapoints and calculated rolling averages as a text file
def file(file_name,data_list,data_point,rolling_average):
    file = open(file_name, 'a') #opens the file
    if len(data_list)== 1: #condition for creating the header
        open(file_name, 'w').close() #clears the file's contents
        file.write("EMG(Raw)" + '\t' + "EMG(Avg)" + '\n' + str(data_point) + '\n') #creates headers
        file.close() #closes the file
    else:
        file.write(str(data_point) + '\t' + str(rolling_average) + '\n') #enters the data points and rolling average
        file.close() #closes the file

***
*****
*** Function Definitions - STUDENT CODE ENDS HERE! ***
*****
***

```


Figure 8B: Screenshot of function definitions of computer program file 5.0 (Part 2).

Comments were added to the file to explain the functions. This file became the template of future files. Buzzer here was used as a local variable and the buzz_text_reminder was refined as well based on previous concepts.

```

*****
Below this line is where you should IMPLEMENT each of the
required tasks.
- DO NOT define any functions in this section!
- You can liken this to a main() function where you
  would implement your individual functions in your
  Lab Mini-Milestones
- PAY ATTENTION TO YOUR INDENTATIONS!!!!
...
## Start with indentation aligned with this comment
sensor = Muscle_Sensor()
data_list = []

while True:
    data_point = sensor.muscle_scaled(50)
    print("Raw Data:", data_point)
    data_list.append(data_point)
    rolling_average = calc_rolling_average(data_list)
    rest_time = calc_rest(rolling_average)
    buzz_text_reminder(rest_time, rolling_average)
    file('testingfile.txt',data_list,data_point,rolling_average)

```

Figure 9B: Screenshot of the implementing section of computer program file 5.0.

The buzzer was moved into the function as a local variable because the program keeps crashing. All functions were called properly within the implementation section.

```

def calc_rolling_average(data_list):
    n_items = 10 #the number of items needed for calculating one average, this value could be changed
    if (len(data_list) >= n_items): #once the number of data points exceeds items, rolling average can be made
        average_list = [] #initializing the list of average numbers
        for i in range(1, (n_items + 1)): #range of values is equal to items
            average_list.append(data_list[-i]) #adds the previous number to the list
        rolling_average = round(sum(average_list)/(n_items),2) #calculates the average, rounded to two decimal places
        return rolling_average #returns the value of the rolling average

#This function is used to calculate the user's rest time, the time interval when they are not meeting the exercise expectations
def calc_rest(rolling_average):
    active_EMG = 30 #this value is used for demonstration, usually its given to the patient from the physician
    rest_time = 0
    time_interval = 10 #this value is used for demonstration, in reality, the resting time interval will be much longer (e.g. 30mins)
    #print("Voltage:", rolling_average) #for demonstrating purposes in design expo
    if rolling_average != None and rolling_average >= active_EMG:
        rest_time = 0
    else:
        while rolling_average != None and rolling_average < active_EMG:
            time.sleep(1)
            rest_time += 1
            if rest_time > time_interval:
                break #allows the code to move on in the main function once the user's resting time exceeds the prescribed resting interval
        return rest_time

#This function will send a message to the physician once it detects abnormal EMG value
import smtplib
from email.mime.text import MIMEText

def notify_physician(rolling_average, user_name):
    normal_EMG = 10
    if rolling_average != None and rolling_average < normal_EMG:
        # connect with Google's servers
        smtp_ssl_host = 'smtp.gmail.com'
        smtp_ssl_port = 465
        # using the email to log in
        username = 'mr.muscles.emg@gmail.com'
        password = 'DP3Team22'

        from_addr = 'mr.muscles.emg@gmail.com' #email sending the message
        to_addrs = ['sri.sharanya22@gmail.com'] #email receiving the message

        # using MIMEText to send text
        message = MIMEText("Please check the patient profile for " + str(user_name))
        message['subject'] = 'Mr.Muscles EMG Sensor'
        message['from'] = from_addr

```

Figure 9B: Screenshot of function definitions of computer program file 6.0 (Part 1).

More detailed comments were added to the functions.

```
def notify_physician(rolling_average,user_name):
    normal_EMG = 10
    if rolling_average != None and rolling_average < normal_EMG:
        # connect with Google's servers
        smtp_ssl_host = 'smtp.gmail.com'
        smtp_ssl_port = 465
        # using the email to log in
        username = 'mr.muscles.emg@gmail.com'
        password = 'DP3Team22'

        from_addr = 'mr.muscles.emg@gmail.com' #email sending the message
        to_addrs = ['sri.sharanya22@gmail.com'] #email receiving the message

        # using MIMEText to send text
        message = MIMEText("Please check the patient profile for " + str(user_name))
        message['subject'] = 'Mr.Muscles EMG Sensor'
        message['from'] = from_addr
        message['to'] = ', '.join(to_addrs)

        # connecting using SSL
        server = smtplib.SMTP_SSL(smtp_ssl_host, smtp_ssl_port)
        # logging in
        server.login(username, password)
        # sending the message
        server.sendmail(from_addr, to_addrs, message.as_string())
        server.quit()

#This function reminds the user with both a buzzer and text message
#Once the user exceeds the resting time_interval, the buzzer turns on and sends a text message to the user's phone(in this case the shell)
#If the user starts exercising again, the buzzer will turn off
def buzz_text_reminder(rest_time,rolling_average):
    buzzer = Buzzer(18)
    sound_duration = 5 #This value could be changed by the user
    time_interval = 10 #This value is used for demonstration, in reality, the resting time interval will be much longer (e.g. 30mins)
    active_EMG = 30 #This value is used for demonstration, usually its given to the patient from the physician
    if rest_time > time_interval:
        buzzer.on()
        time.sleep(sound_duration) #duration of sound
        print("It is time to exercise again.") #The text message is sent to the user's phone
        if rolling_average != None and rolling_average >= active_EMG:
            buzzer.off()
    elif rolling_average != None and rolling_average <= active_EMG: #Maybe change it to while True
        buzzer.on()
        time.sleep(sound_duration) #duration of buzz sound
        print("It is time to exercise again.") #The text message is sent to the user's phone
        buzzer.off()
```

Figure 10B: Screenshot of function definitions of computer program file 6.0 (Part 2).

More detailed comments were added to the code. A new function named notify physician was defined. The purpose of this function is to log in to the Mr. Muscles Gmail and send an email (an actual email) to the physician when the patient has abnormal voltages that are lower than the normal values.

```

server = smtplib.SMTP_SSL(smtp_ssl_host, smtp_ssl_port)
# logging in
server.login(username, password)
# sending the message
server.sendmail(from_addr, to_addrs, message.as_string())
server.quit()

#This functions reminds the user with both a buzzer and text message
#Once the user exceeds the resting time_interval, the buzzer turns on and sends a text message to the user's phone(in this case the shell)
#If the user start exercising again, the buzzer will turn off
def buzz_text_reminder(rest_time,rolling_average):
    buzzer = Buzzer(18)
    sound_duration = 5 #This value could be changed by the user
    time_interval = 10 #This value is used for demonstration, in reality, the resting time interval will be much longer (e.g. 30mins)
    active_EMG = 30 #This value is used for demonstration, usually its given to the patient from the physician
    if rest_time > time_interval:
        buzzer.on()
        time.sleep(sound_duration) #duration of sound
        print("It is time to exercise again.") #The text message is sent to the user's phone
        if rolling_average != None and rolling_average >= active_EMG:
            buzzer.off()
        elif rolling_average != None and rolling_average <= active_EMG: #Maybe change it to while True
            buzzer.on()
            time.sleep(sound_duration) #duration of buzz sound
            print("It is time to exercise again.") #The text message is sent to the user's phone
            buzzer.off()

#This function outputs the input datapoints and calculated rolling averages as a text file
def file(file_name,data_list,data_point,rolling_average):
    file = open(file_name, 'a') #opens the file
    if len(data_list)== 1: #condition for creating the header
        open(file_name, 'w').close() #clears the file's contents
        file.write("EMG(Raw)" + '\t' + "EMG(Avg)" + '\n' + str(data_point) + '\n') #creates headers
        file.close() #closes the file
    else:
        file.write(str(data_point) + '\t' + str(rolling_average) + '\n') #enters the data points and rolling average
        file.close() #closes the file

...
*** Function Definitions - STUDENT CODE ENDS HERE! ***
...

```

Figure 11B: Screenshot of function definitions of computer program file 6.0 (Part 3).

More detailed comments were added to the code as shown.

```

...
## Start with indentation aligned with this comment

sensor = Muscle_Sensor()
data_list = []
user_name = input("Please enter your name: ")
print ("You have successfully setup Mr.Muscles, enjoy your time exercising =^_ ^=!")

while True:
    data_point = sensor.muscle_scaled(200) #The upper limit could be changed by the user/physician
    #print("Raw Data:", data_point) #For demonstrating purposes during design expo
    data_list.append(data_point)
    rolling_average = calc_rolling_average(data_list)
    notify_physician(rolling_average,user_name)
    rest_time = calc_rest(rolling_average)
    buzz_text_reminder(rest_time, rolling_average)
    file('EMGfile.txt',data_list,data_point,rolling_average)

```

Figure 12B: Screenshot of the implementing section of computer program file 6.0.

The program now requires a user input of their name, the patient's name is used when sending an email to the physician alerting them which patient's profile needs to be checked on.


```
def print_block(data_point,rolling_average):
    b_status = 'Off'
    if buzzer.value == 1:
        b_status = 'On'
    reminder_status = 'None'
    if reminder == 1:
        reminder_status = "It is time to exercise again."
    email_status = 'None'
    if email == 1:
        email_status = "Email has been sent to physician."
    print(str(round(data_point,2)) + '\t' + str(rolling_average) + '\t' + b_status + '\t' + reminder_status + '\t' + email_status)
```

Figure 13B: Screenshot of the print block function section of computer program file 7.0.

A new function named the print_block function was created for printing the required information (data_point, rolling_average, output device status) to the shell. This was added because we found out that we missed some instructions when we doubled checked the DP3 Project Module.

```
*****
*** Program Implementation - STUDENT CODE BEGINS HERE! ***
*****
Below this line is where you should IMPLEMENT each of the
required tasks.
- DO NOT define any functions in this section!
- You can liken this to a main() function where you
  would implement your individual functions in your
  Lab Mini-Milestones
- PAY ATTENTION TO YOUR INDENTATIONS!!!!
...
## Start with indentation aligned with this comment

sensor = Muscle_Sensor()
global buzzer
buzzer = Buzzer(18)
data_list = [] #initializes list of data points
user_name = input("Please enter your name: ")#User will enter their name in the shell (acting like a text message)
print ("You have successfully setup Mr.Muscles, enjoy your time exercising =^_^=!")
print("EMG(Raw) \t EMG(Avg) \t Buzzer Status \t User Text \t Physician Email")#header for data + output statuses

while True:
    data_point = sensor.muscle_scaled(200) #The upper limit could be changed by the user/physician
    data_list.append(data_point) #adds the data point from the sensor to the list
    rolling_average = calc_rolling_average(data_list) #calculates rolling average

    rest_time = calc_rest(rolling_average) #calculates time period of rest
    buzz_text_reminder(rest_time, rolling_average) #reminds the user to exercise through a buzzer and text message
    notify_physician(rolling_average,user_name) #notifies physician of EMG values that are below resting values

    print_block(data_point,rolling_average) #prints out the output statuses
    file('EMGfile.txt',data_list,data_point,rolling_average) #creates a text file for examination

...
*****
*** Program Implementation - STUDENT CODE ENDS HERE! ***
*****
```

Figure 14B: Screenshot of the implementing section of the final version of the computer program (8.0).

The buzzer was restated as a global variable here to prevent the program from crashing. A new line of code was added to print the header for data and output statuses. The while loop calls all the defined functions continuously. It includes the rolling_average function, calc_rest function, notify_physician function, print_block function, and finally the file function.

```

***
*****
*** Function Definitions - STUDENT CODE BEGINS HERE! ***
*****
Below this line is where you should DEFINE all functions
- Any output devices that are used in your functions
  should be defined here as well
- PAY ATTENTION TO YOUR INDENTATIONS!!!!
***
## Write your code below this line

#This function is used to calculate the rolling average using the data list of EMG voltage values and output this average as a single data point
def calc_rolling_average(data_list):
    n_items = 5 #the number of items needed for calculating one average, this value could be changed
    if (len(data_list) >= n_items): #once the number of data points exceeds items, rolling average can be calculated
        average_list = [] #initializes the list of average numbers
        for i in range(1, (n_items + 1)): #range of values is equal to number of items
            average_list.append(data_list[-i]) #negative index is used to reference the recent data points
        rolling_average = round(sum(average_list)/(n_items),2) #calculates the average, rounded to two decimal places
        return rolling_average #returns the value of the rolling average

#This function is used to calculate the user's rest time, the time interval when they are not exercising
def calc_rest(rolling_average): #rolling average is passed through as an argument
    global active_EMG
    active_EMG = 30 #This value is used for demonstration, usually its given to the patient from the physician
    rest_time = 0 #default rest time is set to 0 when use is not resting
    global time_interval
    time_interval = 5 #This value is used for demonstration, in reality, the resting time interval will be much longer (e.g. 30mins)
    if rolling_average != None and rolling_average >= active_EMG:
        "For the first 5 rolling averages, the returned value will be none, and the none value cannot be used for comparison."
        rest_time = 0 #if the user is active, their rest time remains 0
    else:
        while rolling_average != None and rolling_average < active_EMG:
            "For the first 5 rolling averages, the returned value will be none, and the none value cannot be used for comparison."
            time.sleep(1)
            rest_time += 1 #when the user is inactive, the rest time is increased by 1 second, until it reaches the maximum time interval
            if rest_time > time_interval:
                break #Allows the code to move on in the main function once the user's resting time exceeds the prescribed resting interval
    return rest_time

```

Figure 15B: Screenshot of the function definitions of the final version of the computer program (8.0 Part 1).

These are final version of the calc_rolling_average function and calc_rest function.

```

#This functions reminds the user with both a buzzer and text message
#Once the user exceeds the resting time_interval, the buzzer turns on and sends a text message to the user's phone(in this case the shell)

#If the user start exercising again, the buzzer will turn off
def buzz_text_reminder(rest_time,rolling_average):
    global reminder #Globalizes the variable so that it can be used in the print function as well
    reminder = 0 #if the user has regular EMG voltage levels, the reminder value = 0
    buzzer.off()
    if rest_time > time_interval:
        "Once the user exceeds the resting time interval that the physician recommend, the buzzer turns on and sends a text message to the user's phone reminding them to exercise(in th
        buzzer.on()
        reminder = 1 #The text message is sent to the user's phone when reminder = 1 in the print function
    if rolling_average != None and rolling_average >= active_EMG:
        "For the first 5 rolling averages, the returned value will be none, and the none value can not be used for comparison."
        "If the user starts exercising again and the voltage value is above the EMG value, the buzzer will turn off."
        buzzer.off()
    else:
        "If the user does not exercising again and the voltage value is still below the active EMG value, the buzzer will turn on again and the text message will be sent as well."
        buzzer.on()
        reminder = 1 #The text message is sent to the user's phone when reminder = 1 in the print function

#This function will send a message to the physician's email once it detects an abnormally low EMG value
import smtplib #Although imports are usually written at the very top of a program, we were told not to touch that code
from email.mime.text import MIMEText

def notify_physician(rolling_average,user_name):
    normal_EMG =10 #This is the threshold value for the EMG to be lower than, in order for this function to be activated
    global email #Globalizes the variable email so that it can be used in the print function
    email = 0 #The value of email = 0 means that no email will be sent
    if rolling_average != None and rolling_average < normal_EMG: #Function is activated when rolling average is lower than a normal resting EMG voltage
        email = 1 #The value of email = 1 indicates that an email will be sent in the shell
        # Connects with Google's servers
        smtp_ssl_host = 'smtp.gmail.com'
        smtp_ssl_port = 465
        # Uses the email to log in
        username = 'mr.muscles.emg@gmail.com'
        password = 'DP3Team22'

        from_addr = 'mr.muscles.emg@gmail.com' #Sets the email sending the message
        to_addrs = ['dr.muscle.atrophy@gmail.com'] #Sets the email receiving the message

        # Uses MIMEText to send text
        message = MIMEText("Please check the patient profile for " + str(user_name))
        message['subject'] = 'Mr.Muscles EMG Sensor'

```

Figure 16B: Screenshot of the function definitions of the final version of the computer program (8.0 Part 2).

These are final version of the buzz_text_reminder function and notify_physician function. The status of the buzzer, email and reminder are initially set to 0 to avoid hard coding.

```

#This function will send a message to the physician's email once it detects an abnormally low EMG value
import smtplib #Although imports are usually written at the very top of a program, we were told not to touch that code
from email.mime.text import MIMEText

def notify_physician(rolling_average,user_name):
    normal_EMG = 10 #This is the threshold value for the EMG to be lower than, in order for this function to be activated
    global email #Globalizes the variable email so that it can be used in the print function
    email = 0 #The value of email = 0 means that no email will be sent
    if rolling_average != None and rolling_average < normal_EMG: #Function is activated when rolling average is lower than a normal resting EMG voltage
        email = 1 #The value of email = 1 indicates that an email will be sent in the shell
        # Connects with Google's servers
        smtp_ssl_host = 'smtp.gmail.com'
        smtp_ssl_port = 465
        # Uses the email to log in
        username = 'mr.muscles.emg@gmail.com'
        password = 'DP3Team22'

        from_addr = 'mr.muscles.emg@gmail.com' #Sets the email sending the message
        to_addrs = ['dr.muscle.atrophy@gmail.com'] #Sets the email receiving the message

        # Uses MIMEText to send text
        message = MIMEText("Please check the patient profile for " + str(user_name))
        message['subject'] = 'Mr.Muscles EMG Sensor'
        message['from'] = from_addr
        message['to'] = ', '.join(to_addrs)

        # Connects to server using SSL
        server = smtplib.SMTP_SSL(smtp_ssl_host, smtp_ssl_port)
        # Logs into email
        server.login(username, password)
        # Sends the message
        server.sendmail(from_addr, to_addrs, message.as_string())
        server.quit()

```

Figure 17B: Screenshot of the function definitions of the final version of the computer program (8.0 Part 3).

This is the final version of the notify_physician function.

```

#This function uses the data points and rolling average and prints them out along with the corresponding statuses of the buzzer, text message, and email in the shell

def print_block(data_point,rolling_average):
    #If the buzzer is on, 'On' will be printed, otherwise it will be off
    active_output = 1 #This variable was used to avoid hard-coding in a conditional statement when each output device is active
    b_status = 'Off'
    if buzzer.value == active_output:
        b_status = 'On'
    #The user will be sent a text message to exercise again based on the buzz_text_reminder function
    reminder_status = 'None'
    if reminder == active_output: #Text is only activated if reminder = 1, which only happens when user has been resting for too long
        reminder_status = "It is time to exercise again." #This statement represents the text that the user would receive
    #If the user has lower than regular EMG voltage levels, email = 1, which will show that an email has been sent
    email_status = 'None'
    if email == active_output:
        email_status = "Email has been sent to physician."
    #The data point, rolling average and the status of each output device will be printed in columns
    print(str(round(data_point,2)) + '\t' + str(rolling_average) + '\t' + b_status + '\t' + reminder_status + '\t' + email_status)

#This function outputs the input data points(raw) and calculated rolling averages as a text file
def file(file_name,data_list,data_point,rolling_average):
    file = open(file_name, 'a') #Opens the file
    if len(data_list)== 1: #Condition for creating the header (only runs once when the emulator starts for the first data point)
        open(file_name, 'w').close() #Clears the file's contents
        file.write("EMG(Raw)" + '\t' + "EMG(Avg)" + '\n' + str(round(data_point,2)) + '\n') #Creates headers
        file.close() #Closes the file
    else:
        file.write(str(round(data_point,2)) + '\t' + str(rolling_average) + '\n') #enters the data points and rolling average
        file.close() #Closes the file

...
*** Function Definitions - STUDENT CODE ENDS HERE! ***
...

```

Figure 18B: Screenshot of the function definitions of the final version of the computer program (8.0 Part 3).

This is the final version of print_block and the file function.

Appendix C: Demonstration of the Computer program

Figure 1C: Screenshot of the GPIO in the Raspberry Pi emulator in which the output device, a buzzer is connected to GPIO 18 and the EMG Sensor is connected as an input device.

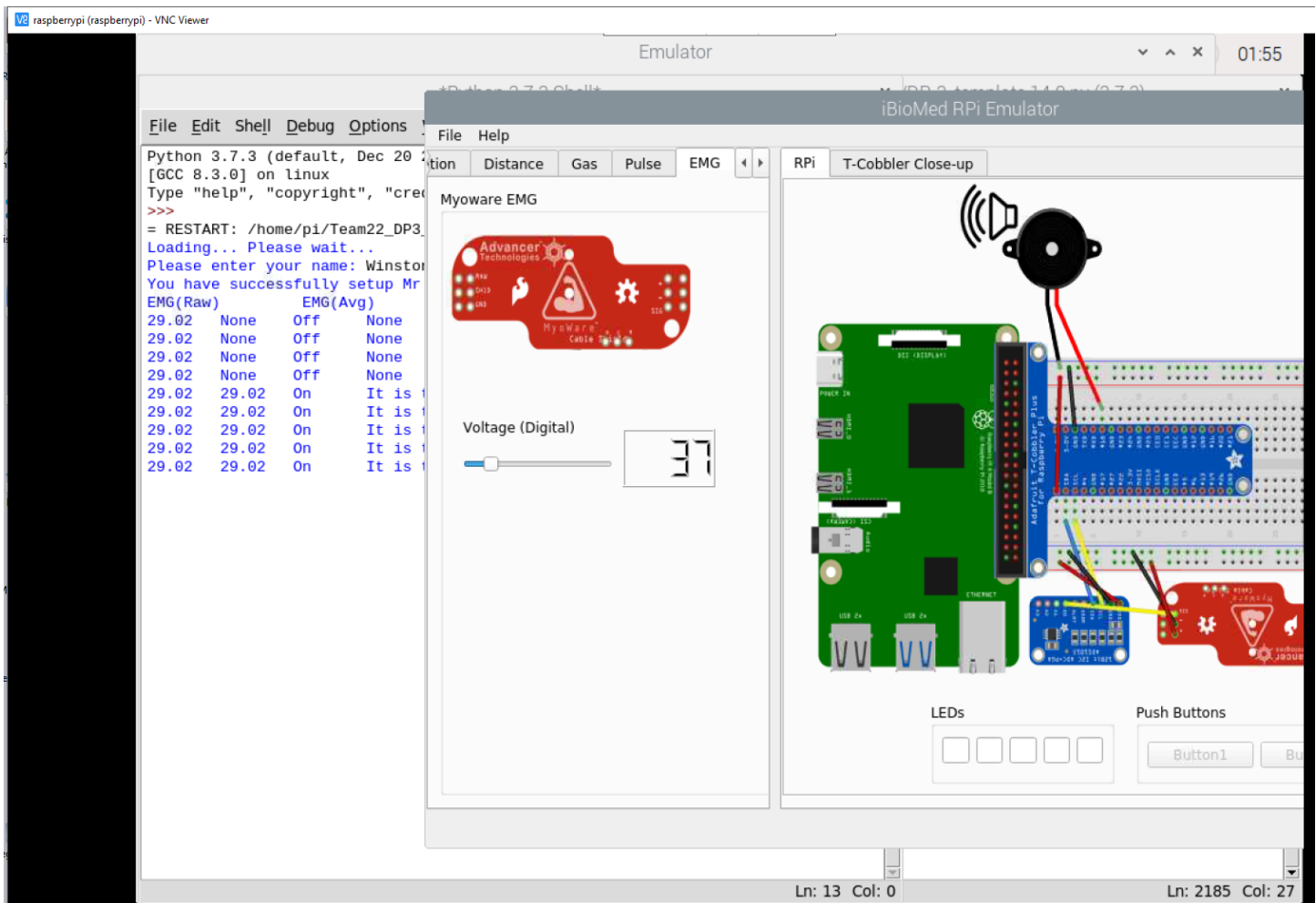


Figure 2C: Screenshot of the Raspberry Pi Emulator showing the buzzer activation as well as the python shell when the program is running.

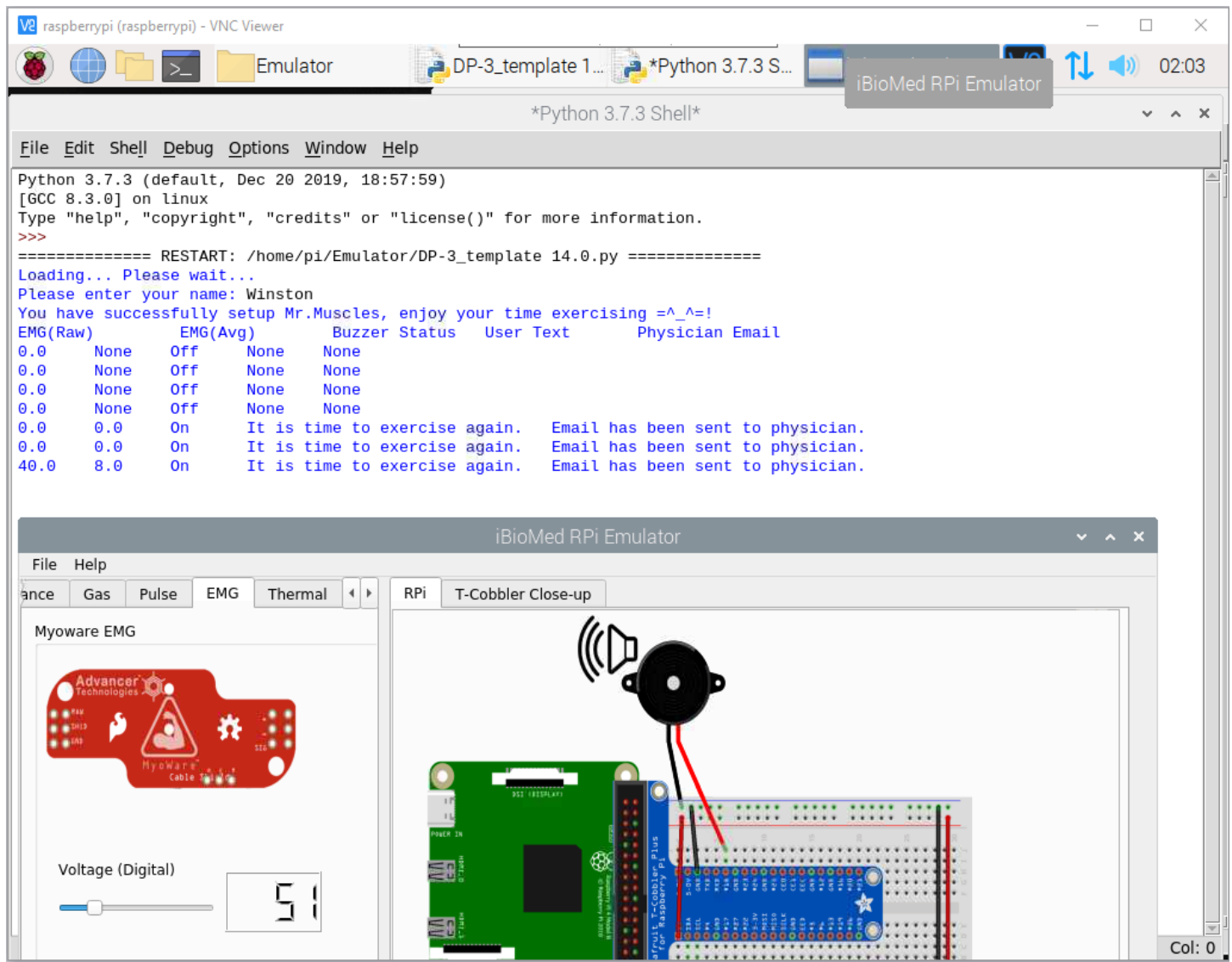


Figure 3C: Screenshot showing all the output device statuses in the python shell after being initialized by the user.

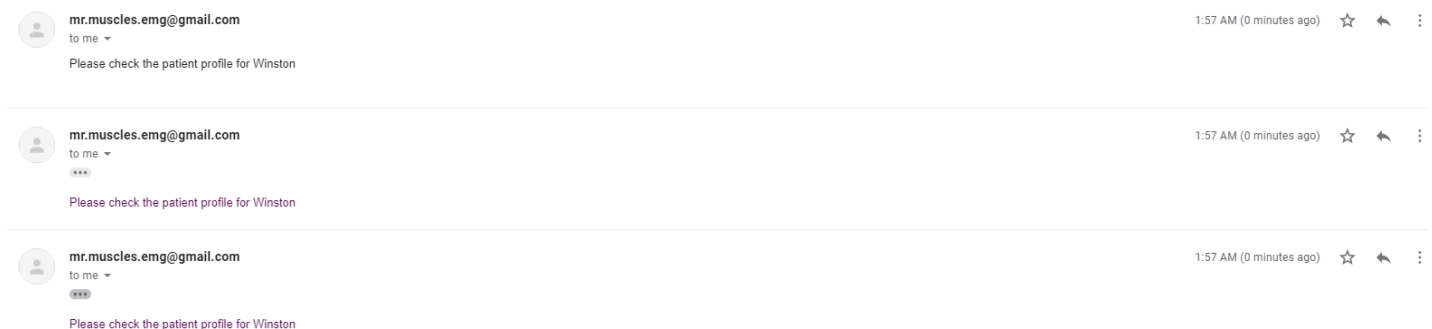


Figure 4C: Screenshot demonstrating the message the physician would receive through email.

Figure 5C: Screenshot of the file that writes the raw and average EMG data when the program is running, and the sensor is in use. This data can be used for examination by the physician.